Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Solution 1

(a) Secant Method for Root of Multiplicity m

The iterative formula for the secant method is given by:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \tag{1}$$

Let r be the exact root. Let $e_n = x_n - r$ be the error in the n-th iteration, so $x_n = r + e_n$.

Assumptions

1. Let r be a root of $f(x)$ with multiplicity $\boldsymbol{m > 1}$.
2. We assume $f(x)$ is m times continuously differentiable ($f \in C^m(I)$) in an interval $\boldsymbol{I = [r - \delta, r + \delta]}$ for some $\delta > 0$. For this derivation, we also assume $f(x)$ is analytic at r, meaning it can be represented by its Taylor series in I.
3. By Taylor's theorem, we expand $f(x)$ about the root r. Since r has multiplicity m, we have $f(r) = f'(r) = \cdots = f^{(m-1)}(r) = 0$ and $f^{(m)}(r) \neq 0$. The Taylor series is:

$$f(x) = f(r) + f'(r)(x-r) + \cdots + \frac{f^{(m-1)}(r)}{(m-1)!}(x-r)^{m-1} + \frac{f^{(m)}(r)}{m!}(x-r)^m + \dots$$
$$f(x) = 0 + 0 + \cdots + 0 + \frac{f^{(m)}(r)}{m!}(x-r)^m + \frac{f^{(m+1)}(r)}{(m+1)!}(x-r)^{m+1} + \dots$$

4. We can factor out $(x-r)^m$ from this series to define a new function $g(x)$:

$$f(x) = (x-r)^m \left[\frac{f^{(m)}(r)}{m!} + \frac{f^{(m+1)}(r)}{(m+1)!}(x-r) + \dots\right] := (x-r)^m g(x) \tag{2}$$

5. Properties of $\boldsymbol{g(x)}$: The function $g(x)$ is defined by the power series in the brackets above.
   - **Non-zero at r:** By evaluating the series for $g(x)$ at $x = r$, all terms except the first (constant) term go to zero:
   $$g(r) = \frac{f^{(m)}(r)}{m!} \neq 0$$
   - **Continuously Differentiable:** A fundamental property of power series is that they are infinitely differentiable (and thus continuously differentiable) inside their radius of convergence. We can find $g'(x)$ by differentiating the series for $g(x)$ term-by-term:
   $$g'(x) = \frac{f^{(m+1)}(r)}{(m+1)!} + 2\frac{f^{(m+2)}(r)}{(m+2)!}(x-r) + 3\frac{f^{(m+3)}(r)}{(m+3)!}(x-r)^2 + \dots$$
   Since $g'(x)$ is also a power series, it is a continuous function in the interval I. Therefore, $g(x)$ is continuously differentiable.

Derivation

Step 1: Expressing the error e_{n+1} in terms of previous errors. We start with the secant formula (1).

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Subtract r on both sides:

$$e_{n+1} = (x_{n+1} - r) = (x_n - r) - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

Substituting $x_k - r = e_k$ and $x_k = e_k + r$:

$$e_{n+1} = e_n - \frac{f(x_n)((r+e_n) - (r+e_{n-1}))}{f(x_n) - f(x_{n-1})}$$
$$e_{n+1} = e_n - \frac{f(x_n)(e_n - e_{n-1})}{f(x_n) - f(x_{n-1})}$$

Taking LCM:

$$e_{n+1} = \frac{e_n(f(x_n) - f(x_{n-1})) - f(x_n)(e_n - e_{n-1})}{f(x_n) - f(x_{n-1})}$$
$$e_{n+1} = \frac{e_n f(x_n) - e_n f(x_{n-1}) - e_n f(x_n) + e_{n-1} f(x_n)}{f(x_n) - f(x_{n-1})}$$

This simplifies to

$$e_{n+1} = \frac{e_{n-1} f(x_n) - e_n f(x_{n-1})}{f(x_n) - f(x_{n-1})} \tag{3}$$

Step 2: Substituting the function definition $f(x) = (x-r)^m g(x)$.

Using (2), we have

$$f(x_n) = (x_n - r)^m g(x_n) = e_n^m g(x_n)$$

and

$$f(x_{n-1}) = e_{n-1}^m g(x_{n-1})$$

Substitute these into (3):

$$e_{n+1} = \frac{e_{n-1}[e_n^m g(x_n)] - e_n[e_{n-1}^m g(x_{n-1})]}{e_n^m g(x_n) - e_{n-1}^m g(x_{n-1})} \tag{4}$$

Step 3: Substituting the Taylor Expansion of $g(x)$ and Analyzing Dominant Terms.

From our assumptions, $g(x)$ is continuously differentiable. We can expand it in a Taylor series about r:

$$g(x_n) = g(r) + g'(r)(x_n - r) + O((x_n - r)^2) = g(r) + g'(r)e_n + O(e_n^2)$$
$$g(x_{n-1}) = g(r) + g'(r)(x_{n-1} - r) + O((x_{n-1} - r)^2) = g(r) + g'(r)e_{n-1} + O(e_{n-1}^2)$$

Substituting these expansions into our error equation (4). Let $C = g(r)$ and $C' = g'(r)$.

$$e_{n+1} = \frac{e_{n-1}e_n^m[C + C'e_n + O(e_n^2)] - e_n e_{n-1}^m[C + C'e_{n-1} + O(e_{n-1}^2)]}{e_n^m[C + C'e_n + O(e_n^2)] - e_{n-1}^m[C + C'e_{n-1} + O(e_{n-1}^2)]} \tag{5}$$

Now, expanding the numerator and denominator:

$$\begin{aligned}
\text{Numerator} &= e_{n-1}e_n^m C + e_{n-1}e_n^m C'e_n + O(e_n^{m+2}) - (e_n e_{n-1}^m C + e_n e_{n-1}^m C'e_{n-1} + O(e_{n-1}^{m+2})) \\
&= C(e_{n-1}e_n^m - e_n e_{n-1}^m) + C'(e_{n-1}e_n^{m+1} - e_n e_{n-1}^{m+1}) + \dots \\
\text{Denominator} &= e_n^m C + e_n^m C'e_n + O(e_n^{m+2}) - (e_{n-1}^m C + e_{n-1}^m C'e_{n-1} + O(e_{n-1}^{m+2})) \\
&= C(e_n^m - e_{n-1}^m) + C'(e_n^{m+1} - e_{n-1}^{m+1}) + \dots
\end{aligned}$$

As the method converges, $n \to \infty$, which means $e_n \to 0$ and $e_{n-1} \to 0$. We can only take the dominant (lowest-order) terms.

- **Numerator:** The first term, $C(e_{n-1}e_n^m - e_n e_{n-1}^m)$, has a total error power of $m+1$. The other terms are of higher order so we can ignore.
- **Denominator:** The first term, $C(e_n^m - e_{n-1}^m)$, has a total error power of m. The other terms are of higher order and less dominant so we are ignoring them also $n \to \infty$.

The dominant term in the numerator is $C(e_{n-1}e_n^m - e_n e_{n-1}^m)$. The dominant term in the denominator is $C(e_n^m - e_{n-1}^m)$.

As $n \to \infty$, the error e_{n+1} is approximated by the dominant terms of numerator and denominator.

$$e_{n+1} \approx \frac{C(e_{n-1}e_n^m - e_n e_{n-1}^m) + \dots}{C(e_n^m - e_{n-1}^m) + \dots} \approx \frac{C(e_{n-1}e_n^m - e_n e_{n-1}^m)}{C(e_n^m - e_{n-1}^m)} \tag{6}$$

Canceling the constant $C = g(r)$ (which is non-zero) from the numerator and denominator:

$$e_{n+1} \approx \frac{e_{n-1}e_n^m - e_n e_{n-1}^m}{e_n^m - e_{n-1}^m} \tag{7}$$

An Approach From Paper By Pedro Díez.

This method is more rigourous and the results match with the paper. Till step 3 it is same the later part part is different and we will do this here.

Step 4: Simplifying the error expression. Factoring the numerator and denominator of the equation (7):

$$e_{n+1} \approx \frac{e_n e_{n-1}(e_n^{m-1} - e_{n-1}^{m-1})}{e_n^m - e_{n-1}^m} = e_n e_{n-1}\left[\frac{e_n^{m-1} - e_{n-1}^{m-1}}{e_n^m - e_{n-1}^m}\right] \tag{8}$$

Step 5: Analyzing the Order of Convergence. To determine the order of convergence, we first test if the method can be superlinear (order $\alpha > 1$) or if it must be linear ($\alpha = 1$).

Proof by Contradiction (Testing Superlinear Convergence)

Assume the method converges with order $\alpha > 1$ (Superlinear). By definition, $|e_n| \approx \lambda |e_{n-1}|^\alpha$. Let's look at the magnitude of e_n relative to e_{n-1} by taking the ratio:

$$\left|\frac{e_n}{e_{n-1}}\right| \approx \frac{\lambda |e_{n-1}|^\alpha}{|e_{n-1}|} = \lambda |e_{n-1}|^{\alpha-1}$$

Since $\alpha > 1$, the exponent $(\alpha - 1)$ is positive. As the method converges ($n \to \infty$), the error $|e_{n-1}| \to 0$. Therefore:

$$\lim_{n\to\infty}\left|\frac{e_n}{e_{n-1}}\right| = \lambda \cdot 0 = 0$$

Mathematically, this implies that e_n becomes **negligible** compared to e_{n-1} (written as $e_n \ll e_{n-1}$). Consequently, in the subtraction terms of Equation (8), the higher powers of e_{n-1} will dominate:

$$e_n^{m-1} - e_{n-1}^{m-1} \approx -e_{n-1}^{m-1} \quad \text{and} \quad e_n^m - e_{n-1}^m \approx -e_{n-1}^m$$

Substituting these approximations back into Equation (8):

$$e_{n+1} \approx e_n e_{n-1}\left[\frac{-e_{n-1}^{m-1}}{-e_{n-1}^m}\right] = e_n e_{n-1}\left[\frac{1}{e_{n-1}}\right] = e_n$$

This result, $e_{n+1} \approx e_n$, implies that the error does not decrease, which contradicts the assumption of convergence.

Conclusion: The method cannot be superlinear. The convergence must be **linear** ($\alpha = 1$).

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Deriving the Asymptotic Error Constant for Linear Convergence

Since the convergence is linear, the ratio of consecutive errors approaches a constant $\lambda \in (0, 1)$, it is greater than 1 then the error would increase with n:

$$\lim_{n\to\infty} \frac{e_n}{e_{n-1}} = \lambda \implies e_n \approx \lambda e_{n-1} \tag{9}$$

We return to Equation (8). We divide the numerator and denominator of the fraction inside the brackets by e_{n-1}^{m-1} and e_{n-1}^{m} respectively to express terms as ratios of $\frac{e_n}{e_{n-1}}$:

$$\begin{aligned}
e_{n+1} &\approx e_n e_{n-1} \frac{e_{n-1}^{m-1}\left[(\frac{e_n}{e_{n-1}})^{m-1} - 1\right]}{e_{n-1}^{m}\left[(\frac{e_n}{e_{n-1}})^{m} - 1\right]} \\
e_{n+1} &\approx e_n e_{n-1} \frac{1}{e_{n-1}} \frac{\lambda^{m-1} - 1}{\lambda^m - 1} \\
e_{n+1} &\approx e_n \frac{\lambda^{m-1} - 1}{\lambda^m - 1}
\end{aligned}$$

Now, divide both sides by e_n:

$$\frac{e_{n+1}}{e_n} \approx \frac{\lambda^{m-1} - 1}{\lambda^m - 1}$$

Since $\frac{e_{n+1}}{e_n} \to \lambda$ as $n \to \infty$, we equate:

$$\lambda = \frac{\lambda^{m-1} - 1}{\lambda^m - 1}$$

Rearranging to solve for λ:

$$\begin{aligned}
\lambda(\lambda^m - 1) &= \lambda^{m-1} - 1 \\
\lambda^{m+1} - \lambda &= \lambda^{m-1} - 1 \\
\lambda^{m+1} - \lambda^{m-1} - \lambda + 1 &= 0
\end{aligned}$$

We can factor this polynomial by grouping terms:

$$\begin{aligned}
\lambda^{m-1}(\lambda^2 - 1) - (\lambda - 1) &= 0 \\
\lambda^{m-1}(\lambda - 1)(\lambda + 1) - (\lambda - 1) &= 0 \\
(\lambda - 1)[\lambda^{m-1}(\lambda + 1) - 1] &= 0
\end{aligned}$$

For the method to converge, we require $e_n < e_{n-1}$, so $\lambda \neq 1$. Thus, we set the bracketed term to zero:

$$\lambda^{m-1}(\lambda + 1) - 1 = 0$$

Or equivalently:

$$\lambda^m + \lambda^{m-1} - 1 = 0 \tag{10}$$

Conclusion

For a root of multiplicity $m > 1$, the Secant method converges **linearly** ($\alpha = 1$). The asymptotic error constant λ is the unique positive real root of the polynomial equation $\lambda^m + \lambda^{m-1} - 1 = 0$. This confirms the results presented in the analysis by Díez .

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Secant Method for Simple Root ($m = 1$)

Assumptions

1. Let r be a **simple root** of $f(x)$. This implies $f(r) = 0$ and $f'(r) \neq 0$.
2. We assume $f(x)$ is twice continuously differentiable ($f \in C^2(I)$) in an interval $\boldsymbol{I = [r - \delta, r + \delta]}$ containing the root.
3. Since $f(x)$ is smooth, we can use Taylor's theorem to expand $f(x)$ about r.

Derivation

Step 1: Expressing the error e_{n+1} in terms of previous errors. Starting with the standard Secant formula and substituting $x_k = r + e_k$ (exactly as derived in the multiple root case), we arrive at the error equation:

$$e_{n+1} = \frac{e_{n-1}f(x_n) - e_n f(x_{n-1})}{f(x_n) - f(x_{n-1})} \tag{11}$$

Step 2: Taylor Expansion of $f(x)$. We expand $f(x_n)$ and $f(x_{n-1})$ about the root r. Since $f(r) = 0$:

$$\begin{aligned} f(x_n) &= f(r) + f'(r)(x_n - r) + \frac{f''(r)}{2}(x_n - r)^2 + O(e_n^3) \\ &\approx f'(r)e_n + \frac{1}{2}f''(r)e_n^2 \\ f(x_{n-1}) &\approx f'(r)e_{n-1} + \frac{1}{2}f''(r)e_{n-1}^2 \end{aligned}$$

Neglecting higher order terms because they are less dominant for very small error values for larger n.
Step 3: Substituting Expansions into the Error Equation. Substituting these approximations into the numerator of (11):

$$\begin{aligned} \text{Numerator} &= e_{n-1}\left[f'(r)e_n + \frac{1}{2}f''(r)e_n^2\right] - e_n\left[f'(r)e_{n-1} + \frac{1}{2}f''(r)e_{n-1}^2\right] \\ &= f'(r)e_{n-1}e_n + \frac{1}{2}f''(r)e_{n-1}e_n^2 - f'(r)e_n e_{n-1} - \frac{1}{2}f''(r)e_n e_{n-1}^2 \\ &= \frac{1}{2}f''(r)e_n e_{n-1}(e_n - e_{n-1}) \end{aligned}$$

Notice that the linear terms $f'(r)e_n e_{n-1}$ cancel out exactly.

Now, substituting into the denominator:

$$\begin{aligned} \text{Denominator} &= \left[f'(r)e_n + \frac{1}{2}f''(r)e_n^2\right] - \left[f'(r)e_{n-1} + \frac{1}{2}f''(r)e_{n-1}^2\right] \\ &= f'(r)(e_n - e_{n-1}) + \frac{1}{2}f''(r)(e_n^2 - e_{n-1}^2) \end{aligned}$$

As $n \to \infty$, the linear term dominates the quadratic term in the denominator. Thus:

$$\text{Denominator} \approx f'(r)(e_n - e_{n-1})$$

Step 4: Simplifying the Error Expression. Combining the numerator and denominator:

$$e_{n+1} \approx \frac{\frac{1}{2}f''(r)e_n e_{n-1}(e_n - e_{n-1})}{f'(r)(e_n - e_{n-1})} \tag{12}$$

The term $(e_n - e_{n-1})$ cancels out (assuming $e_n \neq e_{n-1}$):

$$e_{n+1} \approx \frac{f''(r)}{2f'(r)}e_n e_{n-1} \tag{13}$$

Let $C = \left|\frac{f''(r)}{2f'(r)}\right|$ be the asymptotic error constant.

$$|e_{n+1}| \approx C|e_n||e_{n-1}|$$

Step 5: Determining the Order of Convergence α. Assume the convergence relation $|e_{n+1}| \approx \lambda |e_n|^\alpha$. This implies $|e_n| \approx \lambda |e_{n-1}|^\alpha$, or $|e_{n-1}| \approx \lambda^{-1/\alpha} |e_n|^{1/\alpha}$.

Substitute this into the error relationship:

$$\begin{aligned} |e_{n+1}| &\propto |e_n| \cdot |e_{n-1}| \\ |e_n|^\alpha &\propto |e_n|^1 \cdot |e_n|^{1/\alpha} \\ |e_n|^\alpha &\propto |e_n|^{1+1/\alpha} \end{aligned}$$

Equating the exponents:

$$\alpha = 1 + \frac{1}{\alpha} \quad \Rightarrow \quad \alpha^2 - \alpha - 1 = 0$$

Solving the quadratic equation for α:

$$\alpha = \frac{1 \pm \sqrt{1 - 4(1)(-1)}}{2} = \frac{1 \pm \sqrt{5}}{2}$$

Since the order of convergence must be positive, we take the positive root.

Conclusion

For the Secant method applied to a simple root, the order of convergence is the Golden Ratio:

$$\boldsymbol{\alpha = \frac{1 + \sqrt{5}}{2} \approx 1.618}$$

This indicates **superlinear convergence**, which is faster than the linear convergence ($\alpha = 1$) observed for roots of multiplicity $m > 1$.

Reference:

- Díez, Pedro. "A note on the convergence of the secant method for simple and multiple roots." *Applied Mathematics Letters*, vol. 16, no. 8, 2003, pp. 1211–1215. (For Secant Method Multiple Roots)
- Atkinson, Kendall E. *An Introduction to Numerical Analysis*. 2nd ed., John Wiley & Sons, 1989. (Secant Method Simple Root)

(b) Muller's Method for Simple Root

Muller's method constructs a quadratic polynomial (a parabola), $P(x)$, that passes through the three previous iterates: (x_n, f_n), (x_{n-1}, f_{n-1}), and (x_{n-2}, f_{n-2}). The next iterate, x_{n+1}, is defined as the root of this parabola closest to x_n, meaning $P(x_{n+1}) = 0$.

Assumptions

1. Let r be a **simple root**, which implies $f(r) = 0$ and $f'(r) \neq 0$.
2. We assume the iterates x_n, x_{n-1}, x_{n-2} are in a sufficiently small interval $\boldsymbol{I = [r - \delta, r + \delta]}$ and converge to r.
3. We assume $f(x)$ is three times continuously differentiable, i.e., $\boldsymbol{f \in C^3(I)}$.
4. **The Interpolation Error Formula:** The interpolation error formula for a quadratic polynomial $P(x)$ that interpolates $f(x)$ at the points x_n, x_{n-1}, x_{n-2}. The formula is:

$$f(x) - P(x) = \frac{f'''(\xi)}{6}(x - x_n)(x - x_{n-1})(x - x_{n-2}) \tag{14}$$

where ξ is some point in the interval I containing x and the interpolation points.

From where did this Error Formula come from? Actually I derived this formula in last assignment but anyways. To prove (14), we fix a point $x \in I$ and define a new function $\phi(t)$:

$$\phi(t) = f(t) - P(t) - (f(x) - P(x)) \frac{(t - x_n)(t - x_{n-1})(t - x_{n-2})}{(x - x_n)(x - x_{n-1})(x - x_{n-2})}$$

This function is constructed to have 4 roots:

- $\phi(x_n) = 0$ (since $f(x_n) = P(x_n)$ and the product term is zero)
- $\phi(x_{n-1}) = 0$ (since $f(x_{n-1}) = P(x_{n-1})$ and the product term is zero)
- $\phi(x_{n-2}) = 0$ (since $f(x_{n-2}) = P(x_{n-2})$ and the product term is zero)
- $\phi(x) = 0$ (since $f(x) - P(x) - (f(x) - P(x)) \cdot 1 = 0$)

By applying Rolle's Theorem, since $\phi(t)$ has 4 roots, its derivative $\phi'(t)$ must have at least 3 roots in I. Applying Rolle's Theorem again, $\phi''(t)$ must have at least 2 roots. Applying it a third time, $\phi'''(t)$ must have at least 1 root, which we call ξ, in the interval I.

Now, we compute the third derivative $\phi'''(t)$. Let $K = \frac{f(x)-P(x)}{\prod(x-x_i)}$ (a constant):

$$\phi'''(t) = \frac{d^3}{dt^3}\left(f(t) - P(t) - K \cdot (t - x_n)(t - x_{n-1})(t - x_{n-2})\right)$$

- $\frac{d^3}{dt^3} f(t) = f'''(t)$
- $\frac{d^3}{dt^3} P(t) = 0$, since $P(t)$ is only a quadratic (degree 2) polynomial.
- The product $(t - x_n)\ldots$ is a cubic $t^3 + \ldots$. Its third derivative is $3! = 6$.

So, the third derivative is:

$$\phi'''(t) = f'''(t) - K \cdot 6 = f'''(t) - (f(x) - P(x))\,\frac{6}{(x - x_n)(x - x_{n-1})(x - x_{n-2})}$$

We know $\phi'''(\xi) = 0$ for some $\xi \in I$:

$$0 = f'''(\xi) - (f(x) - P(x))\frac{6}{(x - x_n)(x - x_{n-1})(x - x_{n-2})}$$

Solving for the error $f(x) - P(x)$ gives us the formula in (14).

$$E(x) = \frac{f'''(\xi)}{6}(x - x_n)(x - x_{n-1})(x - x_{n-2})$$

Derivation

Step 1: Finding the value of $P(r)$. $P(x)$, a parabola that passes through the three previous iterates: (x_n, f_n), (x_{n-1}, f_{n-1}), and (x_{n-2}, f_{n-2}). We use the interpolation error formula (14) and evaluate it at the true root $x = r$.

$$f(r) - P(r) = \frac{f'''(\xi_r)}{6}(r - x_n)(r - x_{n-1})(r - x_{n-2})$$

where ξ_r is in the interval containing r, x_n, x_{n-1}, x_{n-2}. We use our assumptions:

- $f(r) = 0$ (since r is the root).
- $e_k = x_k - r$, which means $r - x_k = -e_k$.
- As $x_k \to r$, the value ξ_r must also converge to r, so $f'''(\xi_r) \approx f'''(r)$.

Substituting these into the equation:

$$0 - P(r) \approx \frac{f'''(r)}{6}(-e_n)(-e_{n-1})(-e_{n-2})$$
$$P(r) \approx \frac{f'''(r)}{6} e_n e_{n-1} e_{n-2}$$

Step 2: Relating the new error e_{n+1} to $P(r)$. The next iterate x_{n+1} is a root of the parabola, so $P(x_{n+1}) = 0$. We want to find the new error $e_{n+1} = x_{n+1} - r$. We can write the exact Taylor expansion for the quadratic $P(x)$ about the root r (because $P(x)$ is a quadratic):

$$P(x) = P(r) + P'(r)(x - r) + \frac{P''(r)}{2}(x - r)^2$$

Now, we set $x = x_{n+1}$:

$$P(x_{n+1}) = P(r) + P'(r)(x_{n+1} - r) + \frac{P''(r)}{2}(x_{n+1} - r)^2$$

We know $P(x_{n+1}) = 0$ and $e_{n+1} = x_{n+1} - r$:

$$0 = P(r) + P'(r)e_{n+1} + \frac{P''(r)}{2}e_{n+1}^2$$

As $n \to \infty$, $e_{n+1} \to 0$. This means e_{n+1}^2 is much smaller (less dominant) than e_{n+1} (i.e., $O(e_{n+1}^2)$ is dominated by $O(e_{n+1})$). We can therefore neglect the higher-order term:

$$0 \approx P(r) + P'(r)e_{n+1}$$
$$e_{n+1} \approx -\frac{P(r)}{P'(r)}$$

Step 3: Combining relations and approximating $P'(r)$. We substitute the result from Step 1 into Step 2:

$$e_{n+1} \approx -\frac{\frac{f'''(r)}{6}e_n e_{n-1} e_{n-2}}{P'(r)}$$

Now we must find $P'(r)$. As the iterates x_n, x_{n-1}, x_{n-2} all converge to r, the interpolating parabola $P(x)$ becomes a better and better approximation of the function $f(x)$ in the interval I. Consequently, the derivative of the parabola $P'(r)$ must converge to the derivative of the function $f'(r)$.

$$\lim_{n\to\infty} P'(r) = f'(r)$$

We can use this approximation $P'(r) \approx f'(r)$. This is valid because we assumed r is a **simple root**, so $f'(r) \neq 0$, and we are not dividing by zero.

$$e_{n+1} \approx -\frac{\frac{f'''(r)}{6}e_n e_{n-1} e_{n-2}}{f'(r)}$$

This gives our final error relation between errors:

$$e_{n+1} \approx \left(-\frac{f'''(r)}{6f'(r)}\right) e_n e_{n-1} e_{n-2} \tag{15}$$

Step 4: Solving for the Order of Convergence α. Let $K = \left|-\frac{f'''(r)}{6f'(r)}\right|$ be the asymptotic error constant. The error relation is:

$$|e_{n+1}| \approx K \cdot |e_n||e_{n-1}||e_{n-2}|$$

We assume a convergence relation of the form $|e_{k+1}| \approx \lambda |e_k|^\alpha$. Ignoring the constants λ and K (as they do not affect the exponents), we can establish proportionalities:

1. $|e_{n+1}| \propto |e_n|^\alpha$
2. $|e_n| \propto |e_n|^1$
3. From $|e_n| \propto |e_{n-1}|^\alpha$, we get $|e_{n-1}| \propto |e_n|^{1/\alpha}$
4. From $|e_{n-1}| \propto |e_{n-2}|^\alpha$, we get $|e_{n-2}| \propto |e_{n-1}|^{1/\alpha} \propto \left(|e_n|^{1/\alpha}\right)^{1/\alpha} = |e_n|^{1/\alpha^2}$

Now substitute these four proportionalities back into the error equation (15):

$$|e_{n+1}| \propto |e_n| \cdot |e_{n-1}| \cdot |e_{n-2}|$$
$$|e_n|^\alpha \propto |e_n|^1 \cdot |e_n|^{1/\alpha} \cdot |e_n|^{1/\alpha^2}$$

Combining terms:

$$|e_n|^{\alpha} \propto |e_n|^{1+1/\alpha+1/\alpha^2}$$

For this relation to hold, the exponents must be equal:

$$\alpha = 1 + \frac{1}{\alpha} + \frac{1}{\alpha^2}$$

To solve for α, we multiply the entire equation by α^2:

$$\alpha^3 = \alpha^2 + \alpha + 1$$

This gives the characteristic polynomial for Muller's method:

$$f(\alpha) = \alpha^3 - \alpha^2 - \alpha - 1 = 0$$

Solving this equation using Muller's method (using the code written for muller's method in assignment 2) with the initial guesses $x_0 = 1, x_1 = 1.5, x_2 = 1.8$ with relative error tolerence = 10^{-4}.

Table 1: Convergence to the root of $\alpha^3 - \alpha^2 - \alpha - 1 = 0$

Iteration	**Calculated Root (α)**
1	1.8414605288924317
2	1.8392814462747908
3	1.8392867551312817

So the root of the equation is ≈ 1.839.

The derivative of the equation is

$$f'(\alpha) = 3\alpha^2 - 2\alpha - 1 = 0$$

whose roots are $3, -1/3$. Now $f(3) * f(-1/3) > 0$, meaning the maxima and minima of $f(\alpha)$ both are either greater than 0 or less than 0. Therefore, there is only one real root of $f(\alpha)$ which is ≈ 1.839.

Except Muller's Method the only real root of the equation is exactly 1.8392867 also called Tribonacci Constant calculated using cubic formula.

Conclusion

The order of convergence α for Muller's method on a simple root is ≈ 1.839 which matches with the one written in class ≈ 1.84. This demonstrates a *superlinear* convergence rate.

Reference: For proof of Muller's Method

- Muller, David E. "A Method for Solving Algebraic Equations Using an Automatic Computer." *Mathematical Tables and Other Aids to Computation*, vol. 10, no. 56, 1956, pp. 208–215.

References are available as pdf's online.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Solution 2

Chaotic world of fractals

(a) Mandelbrot Set

The Mandelbrot set is a famous and visually stunning fractal pattern. It is defined in the complex plane. A complex number c is part of the Mandelbrot set if the sequence defined by:

$$z_{n+1} = z_n^2 + c$$

with the starting condition $z_0 = 0$, does not diverge to infinity.

For our purposes (as we can't do infinite iterations), we iterate this formula up to a maximum number of iterations, `max_iter = 100`. We use an "escape condition" to test for divergence: if at any iteration n, the magnitude (or absolute value) of z_n becomes greater than 2 (i.e., $|z_n| > 2$), we stop.

If $|z_n| \leq 2$ for all n up to `max_iter`, we assume the point c is in the set. The visualization is created by plotting the "escape time" n for each point c in a grid.

Implementation

We performed the following steps, to get the fractals pattern:

1. A 1000×1000 grid is defined. The x-axis (real part) spans from -2.5 to 1.0, and the y-axis (imaginary part) spans from -1.5 to 1.5.
2. A function `mandelbrot(c, maxiter)` is implemented. This function takes a complex number c and returns the number of iterations n it took for $|z_n| > 2$. If the loop completes without escaping, it returns `maxiter`.
3. We looped through every point (i, j) in the 1000×1000 grid. Each point is mapped to its corresponding complex number c. The `mandelbrot` function is called for this c, and the resulting escape count n is stored in a 2D array `fractal_image[i, j]`.
4. The 2D array is plotted using `matplotlib.pyplot.imshow`. A colormap is applied to assign different colors based on the escape time (number of iterations).

Corresponding python implementation of Mandelbrot Function

```
def mandelbrot(c, maxiter = 100):
    z = 0 + 0j
    for n in range (maxiter):
        if abs(z) > 2:
            return n
        z = z*z + c
    return maxiter
```

Now defining the grid and then applying mandelbrot function to every point in it. Then storing the number of iterations for every point in a 2-D numpy array.

```
width = 1000
height = 1000

x_min, x_max = -2.5, 1
y_min ,y_max = -1.5, 1.5

x_values = np.linspace(x_min, x_max, width)
y_values = np.linspace(y_min, y_max, height)

fractal_image = np.zeros((height, width))
```

```
for i in range (height):
    for j in range(width):
        real_part = x_values[j]
        imaginary_part = y_values[i]
        c = complex(real = real_part, imag= imaginary_part)
        fractal_image[i, j] = mandelbrot(c,maxiter=100)
```

Visualization

The colormap shows the number of iterations required for divergence. The yellowish central region consists of points that did not diverge within 100 iterations.

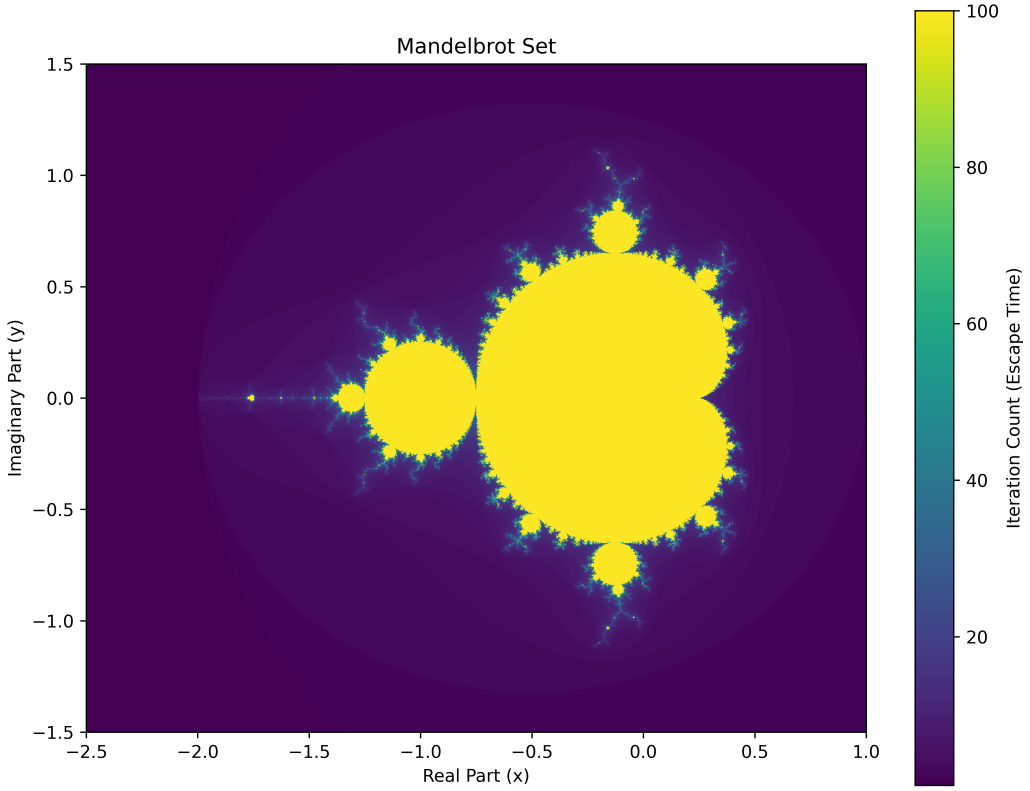

Figure 1: The Mandelbrot set. Generated with $\texttt{max\_iter} = 100$ on a 1000×1000 grid.

Observations

1. We can see most of the points in the grid does not belong to mandelbrot set (purple region). These points diverges under 20 iterations. The yellow region consists of the points which are in mandelbrot set (they do not diverge in 100 iterations).
2. The mandelbrot set consists of distinct geometric shapes connected together.
   (a) A large heart shaped yellow region in the center.
   (b) A circular disk attached to the left of circular disk.
   (c) A lot of small bulbs attached on the boundaries.
   (d) Yellow islands floating near the main body.

3. The boundary of the set is not a smooth line, it is infinitely complex.

4. A fascinating observation is the mandelbrot set exhibits a perfect reflectional symmetry about X-axis ($y = 0$). If you take a complex number $c = a + ib$ its mirror about X-axis is $\bar{c} = a - bi$. Let $\{z_n\}$ be the sequence generated by c, and $\{w_n\}$ be the sequence generated by $\bar{c}$.

   The iteration for the conjugate $\bar{c}$ is:
   $$w_{n+1} = w_n^2 + \bar{c}, \quad w_0 = 0$$

   We can observe that the sequence $\{w_n\}$ is simply the complex conjugate of the sequence $\{z_n\}$. This is because of the two following properties of complex arithmetic:

   (a) The conjugate of a sum is the sum of the conjugates: $\overline{x + y} = \bar{x} + \bar{y}$.
   (b) The conjugate of a square is the square of the conjugate: $\overline{x^2} = (\bar{x})^2$.

   Applying these properties recursively:

   $$\begin{aligned} w_0 &= 0 = \bar{z}_0 \\ w_1 &= 0^2 + \bar{c} = \overline{0^2 + c} = \bar{z}_1 \\ w_2 &= (\bar{z}_1)^2 + \bar{c} = \overline{z_1^2 + c} = \bar{z}_2 \end{aligned}$$

   In general, $w_n = \overline{z_n}$ for all n.

   The determination of whether a point belongs to the Mandelbrot set depends on the magnitude (modulus) of the terms, specifically whether $|z_n|$ exceeds 2. Since a complex number and its conjugate have same magnitude ($|a + bi| = |a - bi|$), it follows that:
   $$|w_n| = |\overline{z_n}| = |z_n|$$

   Therefore, if the sequence for c escapes (diverges) at iteration k, the sequence for $\bar{c}$ will also escape at exactly iteration k. This results in identical behavior for (x, y) and $(x, -y)$, producing an image that is symmetric about the x-axis.

5. Unlike the real axis, the Mandelbrot set is **not** symmetric about the imaginary axis (the y-axis). This occurs because the iterative formula $z_{n+1} = z_n^2 + c$ treats positive and negative real components differently.

   While the z^2 term produces the same result for z and $-z$, the linear addition of $+c$ breaks the symmetry.

   - For a positive real value (e.g., $c = 1$), the term $+c$ adds to the growing square, causing rapid divergence:
     $$0 \to 1 \to 2 \to 5 \ldots \quad \text{(Escapes)}$$
   - For the negative reflection (e.g., $c = -1$), the term $+c$ subtracts from the square, often pulling the value back towards zero and creating stability:
     $$0 \to -1 \to 0 \to -1 \ldots \quad \text{(Stable Loop)}$$

   Hence, the "right" side of the complex plane ($Re(c) > 0$) has less yellow region, while the "left" side ($Re(c) < 0$) contains the bulk of the stable set.

6. Although visually it appears that the isolated islands are disconnected from the main heart shape, the Mandelbrot set is mathematically proven to be connected.

   In 1982, mathematicians Douady and Hubbard proved that all seemingly detached regions are actually joined to the main body by infinitesimally thin filaments. These filaments are often narrower than the sampling resolution of the grid (1000×1000), creating the illusion of disconnection.

7. We can also see a light purple colored circle kind of thing covering the yellow region and outside it, there is dark purple. These light purple region contains the points which diverge after very less iterations. And the dark purple region points diverge even more quickly.

(b) Newton's Fractal

The Newton-Raphson method is a numerical technique for finding roots of equations. This method can also be used to generate a fascinating class of fractals known as Newton's fractals. When applied to complex functions, it can generate fascinating fractal patterns. Here, we find the roots of the polynomial:

$$f(z) = z^3 + 1 = 0$$

This equation has three distinct roots in the complex plane:

$$r_1 = -1, \quad r_2 = e^{i\pi/3}, \quad r_3 = e^{-i\pi/3}$$

We iterate using the Newton-Raphson formula:

$$z_{n+1} = z_n - \frac{f(z_n)}{f'(z_n)} = z_n - \frac{z_n^3 + 1}{3z_n^2}$$

Depending on the starting point z_0, the sequence will converge to one of the three roots. The set of all points z_0 that converge to a specific root forms that root's *basin of attraction*. The boundaries between these basins exhibit a fractal structure.

Implementation

We performed the following steps, to get the fractals pattern:

1. A 1000×1000 grid is defined over the complex plane. Both the real axis (x) and imaginary axis (y) span the interval $[-2, 2]$.
2. A function `newton_fractal(z0, tol)` is implemented. Unlike the Mandelbrot set (which tests for divergence), this function iterates indefinitely until the current value z is within a small tolerance (10^{-6}) of one of the three known roots.
3. We looped through every point (i, j) in the grid. For each point, we ran the Newton iteration. We assigned an integer ID to the point based on the result:
   - 0 if it converged to $r_1 = -1$
   - 1 if it converged to $r_2 = e^{i\pi/3}$
   - 2 if it converged to $r_3 = e^{-i\pi/3}$
4. The resulting 2D array of IDs is plotted using `imshow`. A discrete colormap is used to clearly distinguish the three basins of attraction.

Corresponding python implementation of Newton Fractal Function

```
def newton_fractal(z0, tol = 10**(-6)):
   z = z0
   root_1 = -1
   root_2 = np.exp(1j * np.pi / 3)
   root_3 = np.exp(-1j * np.pi / 3)

   while True:
      if abs(z) == 0:
         return -1
      z = z - (z**3 + 1) / (3*z**2)

      if abs(z - root_1) < tol: return 0
      if abs(z - root_2) < tol: return 1
      if abs(z - root_3) < tol: return 2
```

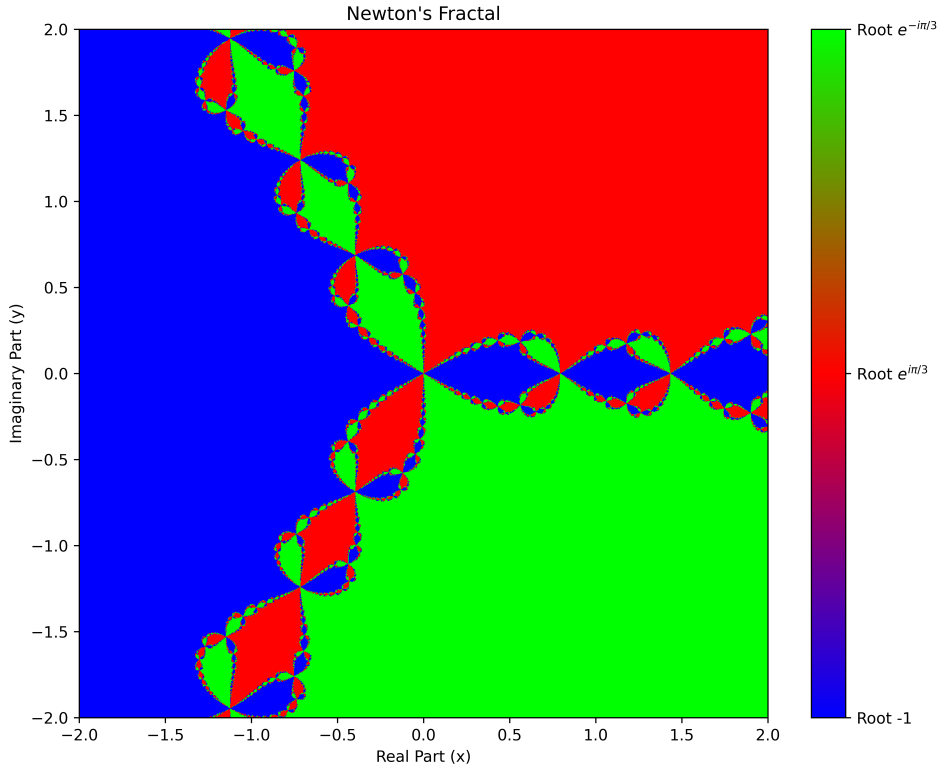

Figure 2: Newton's Fractal showing the basins of attraction for $z^3 + 1 = 0$.

Observations

1. The plot shows three distinct colored regions, representing the basins of attraction for the three roots.
2. The pattern exhibits 3-fold rotational symmetry ($120°$) around the origin, which tell us that there are almost equal number of points (complex numbers) converging to a particular root, consistent with the symmetric placement of the cubic roots on the unit circle.
3. The boundaries between the basins are complex curves called Julia sets (came to know online). Near these boundaries, the iteration is chaotic; a tiny shift in the starting position z_0 can cause the point to converge to a completely different root.
4. A fascinating thing is, wherever two different colors meet, at the boundary the third color is also present. For example, on the boundary between the Blue and Red regions, we can see infinite chains of tiny Green "pearls."
5. This implies that every point on the boundary is in contact with all three basins of attraction simultaneously. One more fascinating thing is how are complex numbers between the two regions converging to the root in region which is opposite to them (like it is converging to the farther root).
6. All three regions meet at origin $(0, 0)$. This point is critical because the derivative of the function is 0 at this point and the newton's method fails here, making the chaotic center of the fractal.
7. Also the root is in the region which converges to that particular root. For example the root -1 lies in the blue region. A point in a region converges to the root of that region only.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

(c) Logistic Map and the Onset of Chaos

The logistic map is a simple yet powerful model used to illustrate how complex, chaotic behavior can arise from a deterministic nonlinear system. Originally introduced as a population growth model, it is defined recursively by:

$$x_{n+1} = Ax_n(1 - x_n), \quad 0 \leq x_n \leq 1$$

where x_n represents the population density at step n, and A is the growth rate parameter.

As A increases, the system behavior changes fundamentally:

- For small A, the sequence settles to a single stable fixed point.
- As A increases, the system undergoes *period-doubling bifurcations* (oscillating between 2 values, then 4, then 8).
- Eventually, the system enters a chaotic regime where the values of x_n appear random.

To quantify this chaos, we calculate the **Lyapunov Exponent** (λ). This value measures the average exponential rate at which nearby trajectories diverge. It is defined as:

$$\lambda_n = \frac{1}{n} \sum_{k=0}^{n-1} \ln |f'(x_k)|$$

Implementation

We performed two specific analyses to visualize this behavior:

1. First we wrote the logistic map function which returns sequence $\{x_n\}$ for the given value of A and initial condition x_0. I took $x_0 = 0.1$
2. Now to understand the sequence for different values of A we made a bifurcation diagram. We varied the control parameter A from 0.89 to 3.995 with an interval of 0.0125. For each A, we iterated the map for 200 steps. To view the long-term behavior, we discarded the first 15 iterations (as they are transient states, the sequence takes some time to normalize) and plotted all remaining values of x. This visualizes the stable points and the branching structure of period-doubling.s
3. We fixed the parameter at $A = 4$ (chaotic regime). We iterated the map for $n = 1000$ steps. At each step, we calculated the local stretching factor $|f'(x)|$ and updated the running average λ_n. We plotted this average against n to observe its convergence.

Corresponding python implementation of logistic_map function

```
def logistic_map(A, x0, n_iter):
sequence = [x0]
for i in range(n_iter):
    x0 = A*x0*(1-x0)
    sequence.append(x0)

return sequence
```

Corresponding python implementation for convergence of lyapunov exponent

```
x0_lyapunov = 0.1
n_iter_lyapunov = 1000
A_lyapunov = 4.0
log_derivative_sum = 0
lyapunov_exponent = []
x = x0_lyapunov
for i in range (1, n_iter_lyapunov + 1):
    derivative = A_lyapunov*(1 - 2*x)
    log_derivative_sum += np.log(abs(derivative))
    lambda_n = log_derivative_sum / i
    lyapunov_exponent.append(lambda_n)
    x = A_lyapunov * x * (1 - x)
```

Results and Observations

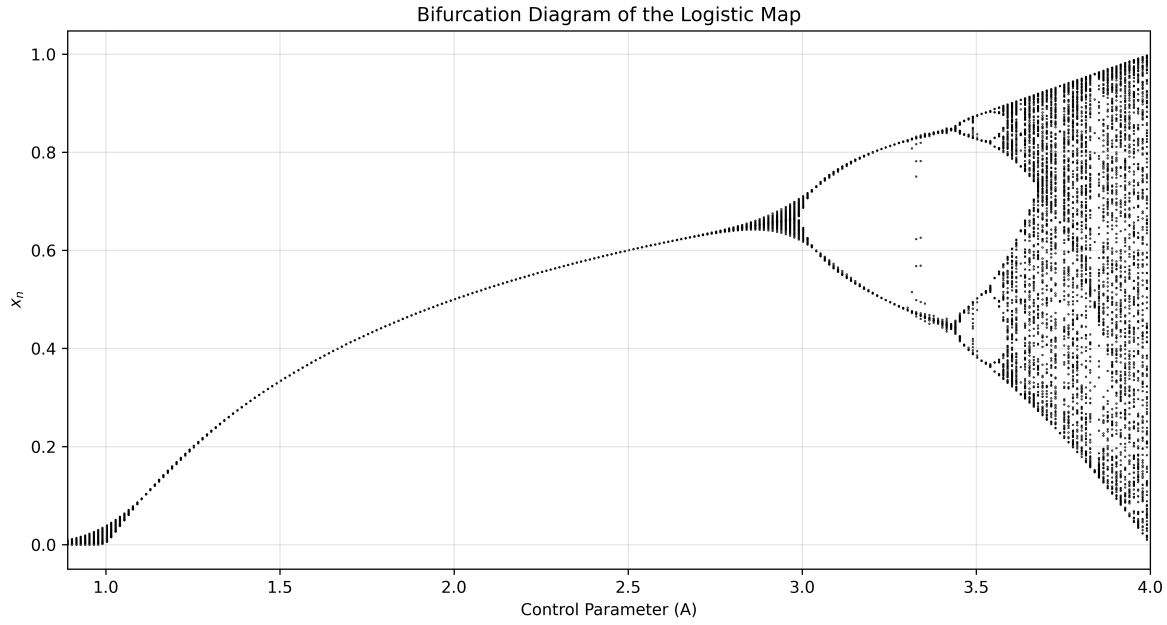

Figure 3: The Bifurcation Diagram. As A increases, the single stable line splits (bifurcates) into 2, then 4, then 8 paths, eventually leading to chaotic bands.

Observations on Bifurcation:

1. In the left portion of the graph (when is in stable state), the system settles into a single stable value. As A increases, this equilibrium state's value (limit of the sequence) gradually rises. Then we see a transition from stable state to chaos.
2. At $A \approx 3.0$, the first bifurcation occurs, and the population oscillates between two values (period-2). This period increases to 4 for A ≈ 3.5, like this is increases.
3. For A $\approx 3.6 - 3.7$, the points fill vertical bands, indicating unpredictability, the terms of the sequence takes random values without forming any pattern, totally chaotic.

So we conclude that, for small values of A the sequence converges to a limit (equilibrium) showing a stable state. For slightly bigger values of A the terms of sequence oscillates between 2 values, on further increasing A the period increases and for more higher values of A the system becomes chaotic, the terms of the sequence are totally random, not following any pattern and spread on a wide region.

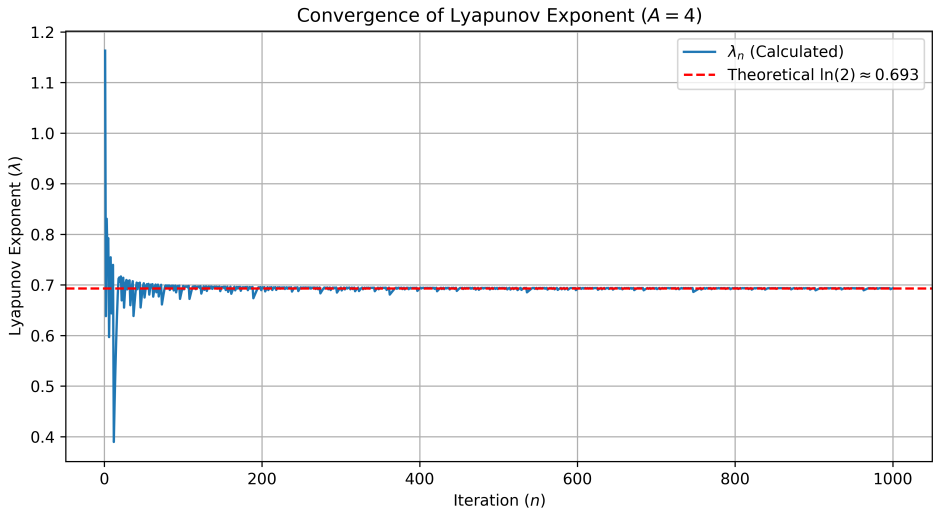

Figure 4: Convergence of the Lyapunov Exponent for $A = 4$. The red dashed line represents the theoretical value $\ln(2)$.

The value of λ_n for large values of n converges to $\ln(2)$.

Interpretation: Stability vs. Chaos

The magnitude and sign of the Lyapunov exponent λ determine the long-term predictability of the system. Consider two parallel simulations of the rabbit population starting with infinitesimally different initial conditions:

$$\text{Universe A: } x_0 \quad \text{and} \quad \text{Universe B: } x_0 + \epsilon$$

where ϵ is a tiny error (e.g., 10^{-9}). The distance between these two populations at step n is approximated by:

$$|\delta x_n| \approx |\delta x_0| e^{\lambda n}$$

- **Scenario 1: Negative Exponent ($\lambda < 0$) $\rightarrow$ Stability**
  If λ is negative, the term $e^{\lambda n}$ approaches zero as $n \to \infty$.
  - **Population Example:** Suppose we set $A = 2.5$. In Universe A, we start with 100 rabbits ($x = 0.1$), and in Universe B, we start with 101 rabbits ($x = 0.101$).
  - **Outcome:** Over time, the difference shrinks. After a few years, both gardens will stabilize at exactly the same population density. The system "forgets" the initial difference. This represents a predictable, non-chaotic system.
- **Scenario 2: Positive Exponent ($\lambda > 0$) $\rightarrow$ Chaos**
  If λ is positive, the term $e^{\lambda n}$ grows exponentially.
  - **Population Example:** Now we set $A = 4.0$. We again start Universe A with 100 rabbits and Universe B with 101 rabbits.
  - **Outcome:** Initially, the populations look similar. However, because errors double at every step (since $\lambda \approx \ln 2$), the tiny difference explodes. After 50 years, Universe A might be facing extinction ($x \approx 0.01$) while Universe B is at maximum capacity ($x \approx 0.99$).
  - **Conclusion:** Long-term prediction is impossible because we can never measure the current population with infinite precision.

Observations on Lyapunov Exponent:

Figure 4 shows the calculated finite-time Lyapunov exponent λ_n as a function of iterations.

1. The plot confirms that for $A = 4$, λ_n converges to a positive constant. A positive exponent is the mathematical definition of chaos, proving that the system exhibits sensitive dependence on initial conditions.
2. The value converges to approximately 0.693. Since $e^{0.693} \approx 2$, this implies that the error between two nearby initial conditions doubles after every iteration. This exponential divergence makes long-term prediction impossible.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Solution 3

Optimizing a Multi-Link Robotic Arm

We consider a planar robotic manipulator consisting of three revolute joints with equal link lengths $l_1 = l_2 = l_3 = 1.0$ m. The configuration of the arm is defined by the joint angles vector $\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3]^T$.

The Cartesian coordinates (x, y) of the end-hook are determined by the forward kinematics equations, derived from the geometry of the arm:

$$x = l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3) \tag{16}$$
$$y = l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3) \tag{17}$$

The objective is to find the optimal joint angles $\boldsymbol{\theta}$ such that the end-hook reaches the fixed target position $\mathbf{p}_t = [-0.8, 1.2]^T$. We can consider this as an optimization problem where we minimize the loss function $L(\boldsymbol{\theta})$, defined as half the squared distance between the current end-hook position $\mathbf{p}(\boldsymbol{\theta})$ and the target $\mathbf{p}_t$:

$$L(\boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{p}(\boldsymbol{\theta}) - \mathbf{p}_t\|^2 \tag{18}$$

To minimize this cost function, we employ iterative gradient-based method.

(a): Derivation of Forward Kinematics Equation

We can see the robotic arm as a sequence of vectors linked end-to-end starting from the origin $(0, 0)$. Let the position of the i-th joint be denoted by (x_i, y_i).

Step 1: First Link (l_1)
The first link is attached to the origin. The angle θ_1 is measured relative to the horizontal axis (x-axis). By taking components, the position of the first joint (x_1, y_1) is the projection of the link length onto the axes:

$$x_1 = l_1 \cos(\theta_1)$$
$$y_1 = l_1 \sin(\theta_1)$$

Step 2: Second Link (l_2)
The second link is attached to the end of the first link. The angle θ_2 provided in the schematic is the *relative* angle between the first link and the second link. Therefore, the *absolute* angle of the second link with respect to the horizontal x-axis is the sum of both the angles: $(\theta_1 + \theta_2)$.

The position of the second joint (x_2, y_2) is obtained by adding the vector components of the second link to the position of the first joint:

$$x_2 = x_1 + l_2 \cos(\theta_1 + \theta_2)$$
$$y_2 = y_1 + l_2 \sin(\theta_1 + \theta_2)$$

Step 3: Third Link (l_3) and End-Hook
Similarly, the third link is attached to the end of the second link. The angle θ_3 is relative to the second link. Consequently, the absolute angle of the third link with respect to the horizontal x-axis is $(\theta_1 + \theta_2 + \theta_3)$.

The final coordinates of the end-hook (x, y) are found by adding the vector components of the third link to the position of the second joint:

$$x = x_2 + l_3 \cos(\theta_1 + \theta_2 + \theta_3)$$
$$y = y_2 + l_3 \sin(\theta_1 + \theta_2 + \theta_3)$$

Step 4: Final Equation
Substituting the expressions for x_1, x_2 and y_1, y_2 into the equations from Step 3, we obtain the complete forward kinematics equations:

$$\begin{aligned} x &= l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3) \\ y &= l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3) \end{aligned} \tag{19}$$

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

(b): Numerical Gradient Computation

To minimize the loss function using Gradient Descent, we require the gradient vector $\nabla L(\boldsymbol{\theta})$. Since the analytical derivative is complex to compute, we approximate it using the **Central Difference Method**.

Deriving from the Taylor series expansion of a function f at step j:

$$f_{j+1} = f_j + hf'_j + h^2\frac{f''_j}{2!} + h^3\frac{f'''(\eta)}{3!} \quad \text{(Forward step)} \tag{20}$$

$$f_{j-1} = f_j - hf'_j + h^2\frac{f''_j}{2!} - h^3\frac{f'''(\zeta)}{3!} \quad \text{(Backward step)} \tag{21}$$

Subtracting the second equation from the first eliminates the quadratic terms:

$$f_{j+1} - f_{j-1} = 2hf'_j + O(h^3) \tag{22}$$

Solving for the first derivative f'_j:

$$f'_j = \frac{f_{j+1} - f_{j-1}}{2h} + O(h^2) \tag{23}$$

This method has a truncation error of order $O(h^2)$, making it more accurate than forward or backward differences.

Applying this to our loss function $L(\boldsymbol{\theta})$ with respect to each parameter θ_i (where $i = 1, 2, 3$) and step size $h = 10^{-5}$, the partial derivatives are:

$$\frac{\partial L}{\partial \theta_1} \approx \frac{L(\theta_1 + h, \theta_2, \theta_3) - L(\theta_1 - h, \theta_2, \theta_3)}{2h} \tag{24}$$

$$\frac{\partial L}{\partial \theta_2} \approx \frac{L(\theta_1, \theta_2 + h, \theta_3) - L(\theta_1, \theta_2 - h, \theta_3)}{2h} \tag{25}$$

$$\frac{\partial L}{\partial \theta_3} \approx \frac{L(\theta_1, \theta_2, \theta_3 + h) - L(\theta_1, \theta_2, \theta_3 - h)}{2h} \tag{26}$$

The full gradient vector is $\nabla L(\boldsymbol{\theta}) = [\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \frac{\partial L}{\partial \theta_3}]^T$.

Corresponding python implementation:

```
l1, l2, l3 = 1.0, 1.0, 1.0
final_position = np.array([-0.8, 1.2])
initial_theta = np.array([0.2, 0.1, -0.3])

alpha = 0.01
beta = 0.9
error = 0.01
h = 10**(-5)

def forward_kinematics(theta):
    t1, t2, t3 = theta[0], theta[1], theta[2]
    x = l1 * np.cos(t1) + l2 * np.cos(t1 + t2) + l3 * np.cos(t1 + t2 + t3)
    y = l1 * np.sin(t1) + l2 * np.sin(t1 + t2) + l3 * np.sin(t1 + t2 + t3)
    return np.array([x, y])

def loss_function(theta):
    current_position = forward_kinematics(theta)
    loss = current_position - final_position
    return 0.5*np.linalg.norm(loss)**2
```

```
def gradient(theta):
    grad = np.zeros_like(theta)
    for i in range(len(theta)):
        theta_plus = theta.copy()
        theta_minus = theta.copy()
        theta_plus[i] += h
        theta_minus[i] -= h
        grad[i] = (loss_function(theta_plus) - loss_function(theta_minus)) / (2 * h)
    return grad
```

Here we first defined the constants like start position, lengths of the arms, final position(target), alpha, beta, error and step size. Then we defined the forward kinematics function which gives us the position of the end hook for a given $\boldsymbol{\theta}$, similarly the loss function. Then for calculating gradient we loop through every θ_i (where $i = 1, 2, 3$) and then apply central difference method to it and store the values in an array.

(c) Gradient Descent Implementation

Gradient Descent is a first-order iterative optimization algorithm used to find a local minimum of a differentiable function. The idea is to take repeated steps in the direction of steepest descent, which is the opposite direction of the gradient. In our problem, we apply this method to iteratively minimize the loss function $L(\boldsymbol{\theta})$.

The algorithm updates the joint angles $\boldsymbol{\theta}$ at each iteration k using the following update rule:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \nabla L(\boldsymbol{\theta}_k) \tag{27}$$

where:

- $\boldsymbol{\theta}_k$ is the vector of joint angles at iteration k.
- $\boldsymbol{\theta}_{k+1}$ is the updated angle vector for the next iteration.
- $\nabla L(\boldsymbol{\theta}_k)$ is the gradient of the loss function at $\boldsymbol{\theta}_k$, which we compute numerically using the Central Difference method defined in part (b).
- α is the **learning rate**, which controls the size of the step taken in the direction opposite to that of the gradient.

Implementation Details

For this problem, we implement the Gradient Descent algorithm with the following parameters and logic:

1. **Initial Conditions:** The optimization starts from the initial joint angles $\boldsymbol{\theta}_0 = [0.2, 0.1, -0.3]^T$ radians.
2. **Learning Rate:** A fixed learning rate of $\alpha = 0.01$ is used, as specified. This value scales the gradient to ensure the updates are small enough to converge stably.
3. **Iterative Process:** The algorithm proceeds in a loop. At each iteration k:
    - The current end-hook position $\mathbf{p}(\boldsymbol{\theta}_k)$ is calculated using the forward kinematics.
    - The Euclidean distance $d = \|\mathbf{p}(\boldsymbol{\theta}_k) - \mathbf{p}_t\|$ is computed.
    - **Stopping Condition:** If $d < 0.01$ m, the end-hook is sufficiently close to the target, and the loop terminates.
    - **Gradient Step:** If the condition is not met, the gradient $\nabla L(\boldsymbol{\theta}_k)$ is computed.
    - **Update Step:** The angles are updated to $\boldsymbol{\theta}_{k+1}$ using the Gradient Descent update rule equation 27.

This process iteratively "walks" the end-hook position towards the target by adjusting the angles $\boldsymbol{\theta}$ down the slope of the loss function.

Corresponding python implementation of the gradient descent algorithm:

```
def gradient_descent(start_theta):
theta = start_theta.copy()
path_x, path_y = [], []
iterations = 0
while True:
    pos = forward_kinematics(theta)
    path_x.append(pos[0])
    path_y.append(pos[1])
    dist = np.linalg.norm(pos - final_position)
    if dist < error:
        break
    grad = gradient(theta)
    theta = theta - alpha * grad
    iterations += 1
return theta, iterations, path_x, path_y
```

(d) Gradient Descent with Momentum Implementation

Gradient Descent with Momentum is an advanced optimization algorithm that improves upon standard Gradient Descent. It introduces a "velocity" vector $\mathbf{v}$ that accumulates past gradients, helping the optimizer to move faster and more stably, especially in complex loss landscapes.

This method is analogous to a ball rolling down a hill. Instead of just following the slope (gradient) at its current position, it gains momentum, allowing it to "roll" through small obstacles or flat regions and accelerate faster on long, consistent slopes.

The algorithm uses two update equations at each iteration k:

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \alpha \nabla L(\boldsymbol{\theta}_k) \tag{28}$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mathbf{v}_{k+1} \tag{29}$$

where:

- $\mathbf{v}_k$ is the velocity vector at iteration k.
- β is the **momentum coefficient**, which determines how much of the previous velocity is retained.
- $\alpha \nabla L(\boldsymbol{\theta}_k)$ is the current gradient's contribution to the velocity.

Key Differences from Standard GD

Standard Gradient Descent (from part c) only uses the current gradient to determine the step: $\Delta\boldsymbol{\theta} = \alpha \nabla L(\boldsymbol{\theta}_k)$.

In Momentum, the step is $\Delta\boldsymbol{\theta} = \mathbf{v}_{k+1}$. The velocity $\mathbf{v}$ is an weighted moving average of past gradients.

- **Why it's faster (fewer iterations):** If the gradient consistently points in the same direction, $\mathbf{v}$ will grow larger and larger, resulting in bigger steps and faster convergence.
- **Why it's more stable:** If the gradient oscillates (e.g., in a narrow "canyon" of the loss function), the positive and negative components of the gradient will partially cancel each other out in the velocity vector. This "dampens" the oscillations and allows the optimizer to follow the main downhill direction more smoothly.

Implementation Details

1. **Initial Conditions:** We start with $\boldsymbol{\theta}_0 = [0.2, 0.1, -0.3]^T$ rad and an initial velocity vector $\mathbf{v}_0 = [0.0, 0.0, 0.0]^T$.
2. **Parameters:** We use the specified learning rate $\alpha = 0.01$ and momentum coefficient $\beta = 0.9$.
3. **Stopping Condition:** The iterative process is identical to standard GD, terminating when the Euclidean distance $\|\mathbf{p}(\boldsymbol{\theta}_k) - \mathbf{p}_t\|$ is less than 0.01 m.
4. **Iterative Process:** At each iteration k, the algorithm performs the following steps:

- The gradient $\nabla L(\boldsymbol{\theta}_k)$ is computed using the Central Difference method.
- The velocity vector is updated by adding a fraction of the previous velocity to the current gradient: $\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \alpha \nabla L(\boldsymbol{\theta}_k)$.
- The joint angles are updated by subtracting the calculated velocity vector: $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mathbf{v}_{k+1}$.

Corresponding python implementation of Momentum Gradient Descent:

```
def gd_momentum(start_theta):
    theta = start_theta.copy()
    velocity = np.zeros_like(theta)
    path_x, path_y = [], []
    iterations = 0

    while True:
        pos = forward_kinematics(theta)
        path_x.append(pos[0])
        path_y.append(pos[1])
        dist = np.linalg.norm(pos - final_position)
        if dist < error:
            break
        grad = gradient(theta)
        velocity = beta * velocity + alpha * grad
        theta = theta - velocity
        iterations += 1
    return theta, iterations, path_x, path_y
```

(e) Results and Discussion

We compared the performance of standard Gradient Descent and Gradient Descent with Momentum in optimizing the robotic arm's joint angles.

1. Numerical Results

Start Position : $(2.93540307, 0.49418954)$.
Target Position : $(-0.8, 1.2)$
The table below summarizes the convergence speed and the final configuration of the arm for both methods.

Metric	**Gradient Descent**	**GD with Momentum**
Iterations to Converge	682	89
Final Position (x, y) **[m]**	$(-0.8086686, 1.20494191)$	$(-0.79717196, 1.20392559)$
Final Angles $[\theta_1, \theta_2, \theta_3]$	$[30.08766456°, 102.32983005°, 49.56716801°]$	$[24.98348912°, 109.95272884°, 40.8386863°]$

Table 2: Performance comparison of optimization methods.

Values accurate upto 6 decimal places. From the values we can see that Gradient Descent with Momentum is more closer to the actual target position.

2. Path Visualization

The end-hook trajectories for both methods are visualized in the figure below.

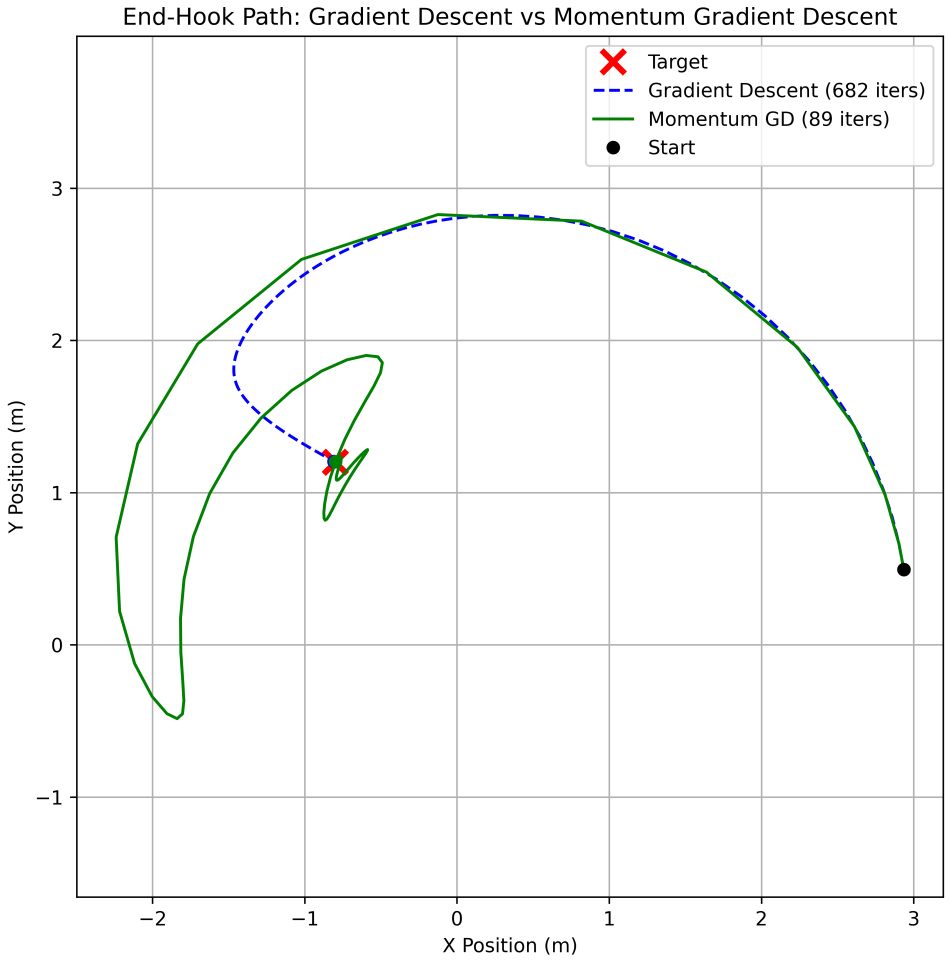

Figure 5: End-hook trajectory: Standard Gradient Descent (Blue Dashed) vs. Momentum (Green Solid).

3. Observations and Discussion

Convergence Speed: Gradient Descent with Momentum converged significantly faster, requiring only **89 iterations** compared to **682 iterations** for standard Gradient Descent. This represents an approximately **7.6x faster**. The Momentum method is faster because it does not only rely on gradient at current position but also takes into the account of the momentum gained so far in the process.

- **Standard Gradient Descent (Blue):** Follows the path of steepest descent strictly. The path is smooth and direct but moves slowly, likely because the gradient of loss function become small in magnitude as we approach to the minima of the loss function.

- **Momentum Gradient Descent(Green):** The path exhibits a large swinging motion. This is due to the momentum term, although if we reach the target position(the minima) approximately, it's momentum is not zero so it moves a little further until the momentum goes to 0 and then it comes back and this process goes on until we met the error condition. So the motion is kind of damped. Even though this method's path looks longer, it converges in less number of steps as this algorithm built's up the velocity and is an accelerated motion it's speed is not constant (as in the standard gradient descent). Hence it converges in less number of steps.

Conclusion: The addition of the momentum term $\beta \mathbf{v}_k$ successfully helped the algorithm overcome small gradients and accelerate convergence. While the path was more oscillatory (swinging wide), the overall time required to reach target position is drastically reduced.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Solution 4

Fourier Series Reconstruction Of Letter Outlines

Any periodic function $z(t)$ with period $T = 1$ can be expressed as an infinite sum of complex exponentials:

$$z(t) = \sum_{k=-\infty}^{\infty} c_k e^{i2\pi kt} \tag{30}$$

where c_k are the Fourier coefficients representing the frequency components of the function. These coefficients are calculated using the integral:

$$c_k = \int_0^1 z(t) e^{-i2\pi kt} dt \tag{31}$$

Since computing an infinite number of coefficients is computationally impossible, we approximate the function using a **Truncated Fourier Series**. By retaining a finite number of modes M, the approximation is given by:

$$z_M(t) = \sum_{k=-M}^{M} c_k e^{i2\pi kt} \tag{32}$$

As the number of modes M increases, the approximation $z_M(t)$ converges to the original shape $z(t)$.

Part 1: Orthogonality Proof [Theoretical]

We have to prove that the set of complex exponential functions $\{e^{i2\pi kt}\}_{k\in\mathbb{Z}}$ is orthogonal with respect to the inner product defined as:

$$\langle f, g \rangle = \int_0^1 f(t)\overline{g(t)}\, dt \tag{33}$$

We will evaluate the inner product $\langle e^{i2\pi kt}, e^{i2\pi mt} \rangle$ for two integers k and m.

Step 1: Let $f(t) = e^{i2\pi kt}$ and $g(t) = e^{i2\pi mt}$. First, we determine the complex conjugate of $g(t)$. Since the variable t is real, the conjugate of the complex exponential is found by negating the exponent:

$$\overline{g(t)} = \overline{e^{i2\pi mt}} = e^{-i2\pi mt}$$

Now, substitute $f(t)$ and $\overline{g(t)}$ into the inner product definition:

$$\begin{aligned} \langle e^{i2\pi kt}, e^{i2\pi mt} \rangle &= \int_0^1 e^{i2\pi kt} \cdot e^{-i2\pi mt}\, dt \\ &= \int_0^1 e^{i2\pi(k-m)t}\, dt \end{aligned}$$

To evaluate this integral, we must consider two distinct cases based on the relationship between k and m.

Step 2: Case 1 ($k = m$). If the integers k and m are equal, then the difference $k - m = 0$. The exponent becomes zero, simplifying the integrand significantly:

$$e^{i2\pi(k-m)t} = e^{i2\pi(0)t} = e^0 = 1$$

Substituting this back into the integral:

$$\langle e^{i2\pi kt}, e^{i2\pi kt} \rangle = \int_0^1 1\, dt = \Big[t\Big]_0^1 = 1 - 0 = 1$$

Thus, the inner product is 1 when the indices are identical.

Step 3: Case 2 ($k \neq m$). If the integers are distinct, let $n = k - m$, where n is a non-zero integer ($n \in \mathbb{Z}, n \neq 0$). The integral becomes:

$$I = \int_0^1 e^{i2\pi nt}\, dt$$

Using the standard integration of exponentials $\int e^{at}dt = \frac{1}{a}e^{at}$:

$$I = \left[\frac{1}{i2\pi n}e^{i2\pi nt}\right]_0^1 = \frac{1}{i2\pi n}\left(e^{i2\pi n(1)} - e^{i2\pi n(0)}\right)$$

We know that $e^0 = 1$. Furthermore, by Euler's identity ($e^{i\theta} = \cos\theta + i\sin\theta$), for any integer n:

$$e^{i2\pi n} = \cos(2\pi n) + i\sin(2\pi n) = 1 + i(0) = 1$$

Substituting these values into the expression:

$$I = \frac{1}{i2\pi n}(1 - 1) = 0$$

Thus, the inner product is 0 when the indices are different.

Conclusion: Combining the results from Step 2 and Step 3, we have proven the orthogonality condition:

$$\langle e^{i2\pi kt}, e^{i2\pi mt}\rangle = \begin{cases} 1, & \text{if } k = m \\ 0, & \text{if } k \neq m \end{cases}$$

This confirms that the set $\{e^{i2\pi kt}\}_{k\in\mathbb{Z}}$ forms an orthonormal system on the interval $[0, 1]$.

Part 2: Data Loading and Preprocessing

We acquired the discrete coordinate samples for the letter 'S' using the provided generator tool. The dataset consists of N pairs of real coordinates (x_j, y_j), which were converted into the complex plane representation:

$$z_j = x_j + i \cdot y_j, \quad \text{for } j = 0, 1, \ldots, N-1 \tag{34}$$

To parameterize the curve, we assigned a time stamp $t_j = j/N$ to each point, mapping the index j to the interval $[0, 1)$.

Part 3: Fourier Coefficient Calculation

Here we compute the Fourier coefficients c_k defined by the integral:

$$c_k = \int_0^1 z(t)e^{-i2\pi kt}\, dt \tag{35}$$

where $z(t)$ is the complex representation of the letter outline parameterized by $t \in [0, 1]$.

Method: Composite Trapezoidal Rule

To evaluate this integral numerically, we will use the **Composite Trapezoidal Rule**. According to this rule, for a function $f \in C^2[a, b]$ and step size $h = (b - a)/n$, the rule is defined as:

$$\int_a^b f(x)dx = \frac{h}{2}\left[f(a) + 2\sum_{j=1}^{n-1} f(x_j) + f(b)\right] - \frac{b-a}{12}h^2 f''(\mu) \tag{36}$$

where the last term represents the truncation error for some $\mu \in (a, b)$.

Derivation for Periodic Contours:
In our specific case:

- The interval is $[a, b] = [0, 1]$.
- The step size is $h = \Delta t = \frac{1}{N}$.
- The integrand is $f(t) = z(t)e^{-i2\pi kt}$.

Since the letter outline is a closed loop, the function $z(t)$ is **periodic** with period $T = 1$. Consequently, the start and end points are identical:

$$f(0) = z(0)e^0 = z(0) \quad \text{and} \quad f(1) = z(1)e^{-i2\pi k} = z(1)$$

Since $z(0) = z(1)$, it follows that $f(0) = f(1)$. Substituting this into the standard formula allows us to combine the boundary terms:

$$\frac{h}{2}[f(0) + f(1)] = \frac{h}{2}[2f(0)] = hf(0)$$

This simplification reduces the Trapezoidal approximation to a standard Riemann sum:

$$c_k \approx \frac{1}{N} \sum_{j=0}^{N-1} z(t_j) e^{-i2\pi k t_j} \tag{37}$$

This is the formula implemented in the code as :

```
def fourier_coefficients(z, t, M):
coeffs = {}
# N is the number of sample points
N = len(z)
# Step size dt = 1/N (since total period T=1)
dt = 1.0 / N
for k in range(-M, M + 1):
    integrand = z * np.exp(-1j * 2 * np.pi * k * t) # integrand is an array
    c_k = np.sum(integrand) * dt
    coeffs[k] = c_k
return coeffs
```

Here z is the array containing the complex numbers, t is the array of time stamps and M is the Mode number.

Justification for Numerical Integration Method (Composite Trapezoidal Rule)

1. For a general smooth function on a non-periodic interval, the global error for the composite trapezoidal rule is

   $$E = O(h^2), \qquad h = \frac{1}{N},$$

   which is lower order compared to Simpson's rule ($O(h^4)$). However, this standard error bound does *not* apply to periodic functions. In our application, the integrand

   $$z(t)\, e^{-i2\pi kt}$$

   is **periodic on** $[0, 1]$ because both $z(t)$ and $e^{-i2\pi kt}$ are periodic. For smooth periodic functions, the composite trapezoidal rule has a property known as **spectral accuracy**, meaning that the error decreases *faster than any polynomial order* and, in fact, decays exponentially.

   To understand why, we use the Euler–Maclaurin expansion of the trapezoidal rule error:

   $$E = \sum_{n=1}^{\infty} c_{2n}\, h^{2n} \big(f^{(2n-1)}(1) - f^{(2n-1)}(0) \big).$$

   For periodic functions, all endpoint derivatives match:

   $$f^{(m)}(0) = f^{(m)}(1) \quad \text{for all integers } m.$$

   Hence *each term in the error expansion vanishes*, giving

   $$E = 0 + 0 + 0 + \cdots = O(e^{-cN}),$$

   where $c > 0$ is a constant depending on the smoothness of f. This exponential decay of error is what is meant by **spectral (or exponentially fast) convergence**.

   Thus the trapezoidal rule becomes significantly more accurate than Simpson's rule for periodic integrands.

2. For N uniformly spaced samples

$$t_j = \frac{j}{N}, \qquad j = 0, 1, \ldots, N-1,$$

the composite trapezoidal rule simplifies to the direct Riemann sum:

$$c_k \approx \frac{1}{N} \sum_{j=0}^{N-1} z(t_j)\, e^{-i2\pi k t_j}.$$

This makes the method computationally simple, vectorizable, and efficient for implementation in Python.

Part 4: Curve Reconstruction

To reconstruct the continuous outline of the letter from the discrete frequency data, we utilize the Fourier coefficients calculated in Part 3. The reconstruction is performed using the **Truncated Fourier Series**, also known as the Synthesis Equation:

$$z_M(t) = \sum_{k=-M}^{M} c_k e^{i2\pi kt} \tag{38}$$

where M represents the number of frequency modes retained (the bandwidth of the reconstruction).

Implementation Details:

- We implement a function that sums these complex exponentials for each time step t.
- To ensure the reconstructed curve appears smooth and continuous (rather than just reproducing the original discrete points), we evaluate this summation on a dense grid of $N_{dense} = 1000$ equally spaced points in the interval $t \in [0, 1)$.
- As the mode number M increases, the approximation $z_M(t)$ converges closer to the original shape $z(t)$, capturing finer details and sharper changes in curvature.

Part 5: Visualization of Convergence

To visualize the improvement in reconstruction accuracy with increasing frequency components, we generated a figure with five subplots comparing the reconstructions for mode numbers $M \in \{2, 8, 32, 64, 128\}$.

The results are displayed in Figure 6, where the original letter outline is shown in gray and the Fourier reconstruction is overlaid in red.

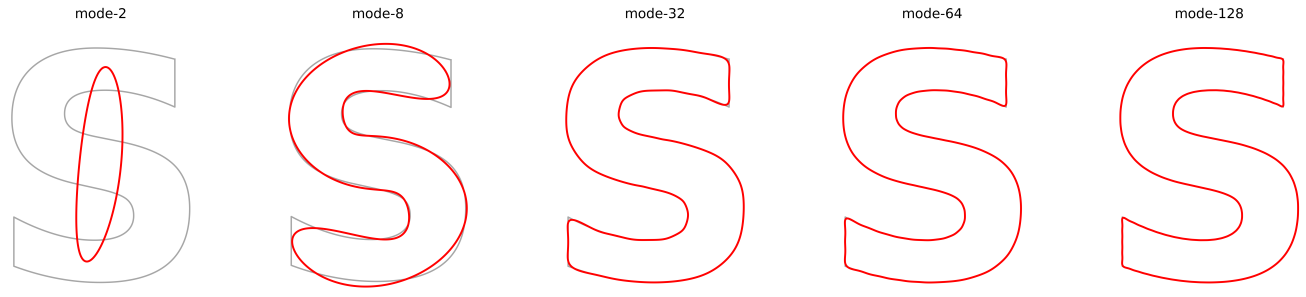

Figure 6: Fourier series reconstruction of the letter 'S' with increasing mode number M. Low modes capture the coarse shape, while high modes resolve fine details.

Observations:

- **Low Modes ($M = 2, 8$):** The reconstruction acts as a low-pass filter, capturing only the fundamental geometric structure (position and general elongation). The result is a smooth, elliptical blob that lacks distinct corners. These try to take the shape and match shape-wise but they don't cover exactly the letter.

- **Medium Modes ($M = 32, 64$):** The characteristic shape of the letter 'S' becomes clearly recognizable. The curvature is well-approximated, though sharp turns still appear rounded due to the truncation of high-frequency components.
- **High Modes ($M = 128$):** The reconstruction is visually indistinguishable from the original curve. The inclusion of higher-order coefficients allows the series to resolve sharp features and corners effectively. But if you zoom then you can see i does not cover some corners.

Part 6: Animation

To dynamically visualize the Fourier synthesis process, we generated an animation that displays the time-evolution of the reconstruction components.

Methodology: The animation represents the truncated Fourier series as a chain of rotating complex vectors. For a set of coefficients $\{c_k\}$ and a time parameter $t \in [0, 1]$, the position of the curve $z(t)$ is the vector sum of individual frequency components:

$$z(t) \approx \sum_{k=-M}^{M} c_k e^{i2\pi kt} \tag{39}$$

- **Rotating Vectors (The Arm):** Each segment of the blue arm corresponds to a specific frequency term $c_k e^{i2\pi kt}$. The length of the segment is $|c_k|$ and its rotation angle is determined by $2\pi kt$.
- **Pen Tip:** The red dot at the end of the chain represents the cumulative sum $z_M(t)$.
- **Trace:** As the animation progresses through $t : 0 \to 1$, the tip traces the reconstructed letter 'S' (shown in yellow).

This visualization demonstrates how the superposition of simple circular motions can generate complex, non-convex planar shapes. The final animation was generated using $M = 128$ modes to ensure high accuracy.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Solution 5

Damped, Driven Harmonic Oscillator

The motion of the damped, driven harmonic oscillator is governed by the second-order ordinary differential equation (ODE) derived from Newton's second law:

$$\frac{d^2x}{dt^2} + 2\gamma\frac{dx}{dt} + \omega_0^2 x = A_0\cos(\omega t) \tag{40}$$

where:

- $x(t)$ is the displacement from equilibrium.
- γ is the damping coefficient.
- ω_0 is the natural angular frequency of the undamped system.
- $A_0\cos(\omega t)$ is the external driving force per unit mass.

Problem Parameters

The simulation is performed over the time interval $t \in [0, 30]$ seconds with the following parameters:

- Natural angular frequency: $\omega_0 = 2\pi$ rad/s
- Damping coefficient: $\gamma = 0.5$ s^{-1}
- Driving amplitude: $A_0 = 2.0$ m/s^2
- Driving frequency: $\omega = 2.0$ rad/s

Initial Conditions

The system starts with the following initial state:

$$x(0) = 1.0\,\text{m}$$
$$v(0) = \left.\frac{dx}{dt}\right|_{t=0} = 0.0\,\text{m/s}$$

Mathematical Formulation

Numerical methods like Euler's method and the Runge-Kutta method are designed to solve systems of first-order ODEs. Therefore, we must reduce the given second-order ODE into a system of two first-order equations.

We define the state variables as displacement x and velocity v:

$$x_1(t) = x(t) \tag{41}$$
$$x_2(t) = v(t) = \frac{dx}{dt} \tag{42}$$

Differentiating these variables with respect to time yields the system:

$$\frac{dx_1}{dt} = x_2 \tag{43}$$
$$\frac{dx_2}{dt} = A_0\cos(\omega t) - 2\gamma x_2 - \omega_0^2 x_1 \tag{44}$$

This system can be written in vector form $\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y})$, where $\mathbf{y} = [x, v]^T$. This formulated system is solved using Euler's Method and the Fourth-Order Runge-Kutta (RK4) method.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Part 1: Numerical Implementation

In this section, we implement two numerical methods to solve the system of first-order ODEs derived in the previous section. We use a time step of $\Delta t = 0.01$ s over the interval $t \in [0, 30]$ s.

Method 1: Euler's Method

Algorithm Description : Euler's method is the simplest numerical integration technique for solving initial value problems. It is derived from the Taylor series expansion of the function $y(x)$ about a point x_n:

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2!}y''(\zeta_n) \tag{45}$$

where h is the step size. By dropping the higher-order error terms (truncation error), we obtain the iterative formula:

$$y_{n+1} = y_n + hf(x_n, y_n) \tag{46}$$

Geometrically, this method approximates the curve using the tangent line at the current point (x_n, y_n) to estimate the value at the next point.

For our system of two variables (x and v), the update equations are:

$$x_{n+1} = x_n + \Delta t \cdot v_n \tag{47}$$

$$v_{n+1} = v_n + \Delta t \cdot \left(A_0 \cos(\omega t_n) - 2\gamma v_n - \omega_0^2 x_n\right) \tag{48}$$

Corresponding Python implementation of the Euler algorithm:

```
def derivatives(t, x, v):
    dxdt = v
    dvdt = A_0*np.cos(W*t) - 2 * gamma * v - (w_0**2) * x
    return dxdt, dvdt

def euler(t_start, t_end, dt, x0, v0):
    n_steps = int((t_end - t_start) / dt) + 1
    t = np.linspace(t_start, t_end, n_steps)
    x = np.zeros(n_steps)
    v = np.zeros(n_steps)

    # Initial conditions
    x[0] = x0
    v[0] = v0


    for i in range(n_steps - 1):
        t_i = t[i]
        x_i = x[i]
        v_i = v[i]

        # Calculate derivatives
        dxdt, dvdt = derivatives(t_i, x_i, v_i)

        # Update next state
        x[i+1] = x_i + dxdt * dt
        v[i+1] = v_i + dvdt * dt

    return t, x, v
```

Here the derivatives function calculates the derivative for given time, displacement, velocity. Then in the euler function first we divide the time in the intervals of $0.01s$ and then we make two arrays for displacement and velocity and set the initial values then perform the Euler's method.

Method 2: Fourth-Order Runge-Kutta (RK4) Method

Algorithm Description : The Euler method requires a very small step size for meaningful results because it only uses the slope at the beginning of the interval. To achieve higher accuracy without evaluating higher-order partial derivatives, we use the Runge-Kutta methods.

The Fourth-Order Runge-Kutta (RK4) method uses a weighted average of four slopes (k_1, k_2, k_3, k_4) calculated within the time step to update the solution. For a system of equations, the update formula is generalized as follows:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{49}$$

where the slopes for our system are:

- $k_1 = \Delta t \cdot \mathbf{f}(t_n, \mathbf{y}_n)$ (Slope at the beginning)
- $k_2 = \Delta t \cdot \mathbf{f}(t_n + \frac{\Delta t}{2}, \mathbf{y}_n + \frac{k_1}{2})$ (Slope at midpoint using k_1)
- $k_3 = \Delta t \cdot \mathbf{f}(t_n + \frac{\Delta t}{2}, \mathbf{y}_n + \frac{k_2}{2})$ (Slope at midpoint using k_2)
- $k_4 = \Delta t \cdot \mathbf{f}(t_n + \Delta t, \mathbf{y}_n + k_3)$ (Slope at the end)

Python Implementation

Corresponding Python implementation of the RK4 algorithm:

```
def rk4(t_start, t_end, dt, x0, v0):
    n_steps = int((t_end - t_start) / dt) + 1
    t = np.linspace(t_start, t_end, n_steps)
    x = np.zeros(n_steps)
    v = np.zeros(n_steps)

    x[0] = x0
    v[0] = v0

    for i in range(n_steps - 1):
        t_i = t[i]
        x_i = x[i]
        v_i = v[i]

        # Calculating slopes
        k1_x, k1_v = derivatives(t_i, x_i, v_i)
        k2_x, k2_v = derivatives(t_i + 0.5*dt, x_i + 0.5*k1_x*dt, v_i + 0.5*k1_v*dt)
        k3_x, k3_v = derivatives(t_i + 0.5*dt, x_i + 0.5*k2_x*dt, v_i + 0.5*k2_v*dt)
        k4_x, k4_v = derivatives(t_i + dt, x_i + k3_x*dt, v_i + k3_v*dt)

        # Weighted average
        x_slope = (k1_x + 2*k2_x + 2*k3_x + k4_x) / 6.0
        v_slope = (k1_v + 2*k2_v + 2*k3_v + k4_v) / 6.0

        # Updating state
        x[i+1] = x_i + x_slope * dt
        v[i+1] = v_i + v_slope * dt

    return t, x, v
```

Here we first do the same thing dividing the intervals and putting initial conditions and then we parallelly calculate the slopes for both x and v and update the state.

Tabulated Results

The displacement $x(t)$ and velocity $v(t)$ were computed using both methods. The results are tabulated below at every 500^{th} time step.

Table 3: Numerical Solution using Euler's Method ($\Delta t = 0.01$ s)

Step	Time (s)	Displacement (m)	Velocity (m/s)
0	0.00	1.000000	0.000000
500	5.00	0.158295	0.034752
1000	10.00	0.071181	-0.110245
1500	15.00	0.015637	0.108838
2000	20.00	-0.033052	-0.088063
2500	25.00	0.053948	0.034068
3000	30.00	-0.054385	0.029507

Table 4: Numerical Solution using RK4 Method ($\Delta t = 0.01$ s)

Step	Time (s)	Displacement (m)	Velocity (m/s)
0	0.00	1.000000	0.000000
500	5.00	0.027607	0.103876
1000	10.00	0.031958	-0.092126
1500	15.00	0.006025	0.112985
2000	20.00	-0.035079	-0.087862
2500	25.00	0.053397	0.035612
3000	30.00	-0.054485	0.028224

We can see that after 30 seconds the displacement and velocity is approximately same from both the methods.

Part 2: Visualization of Solution

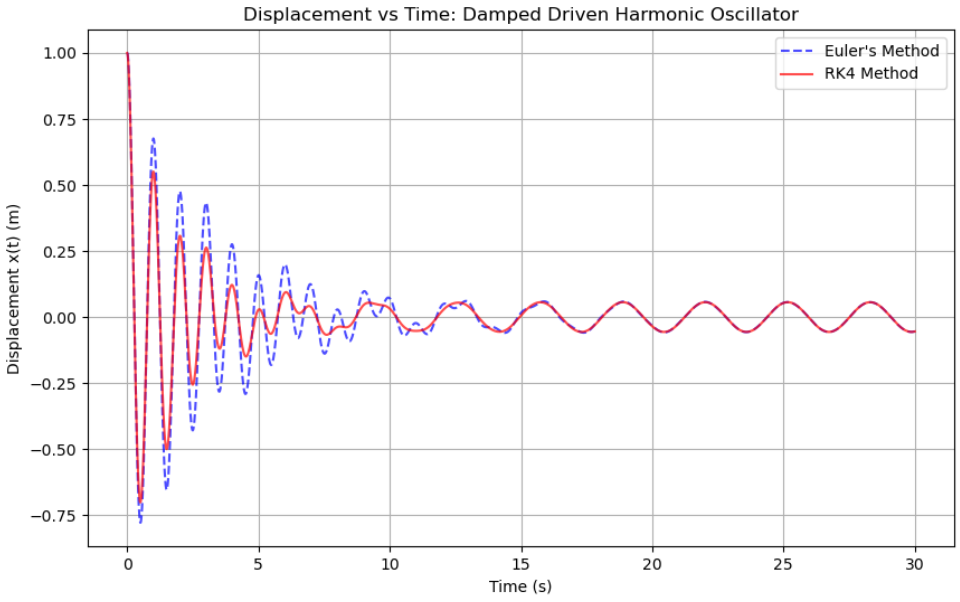

Figure 7: Displacement $x(t)$ vs. Time t for the Damped, Driven Harmonic Oscillator. Comparison between Euler's method and the RK4 method with $\Delta t = 0.01$ s.

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

The numerical solutions for the displacement $x(t)$ obtained using Euler's method and the Fourth-Order Runge-Kutta (RK4) method are visualized in Figure 7.

The plot displays the evolution of the oscillator's motion over the full time interval $t \in [0, 30]$ seconds. The RK4 method (solid red line) provides a more accurate approximation of the system's behavior, while Euler's method (dashed blue line) shows a slight deviation, particularly in the transient phase, due to its lower order of accuracy.

Part 3: Analysis of Motion

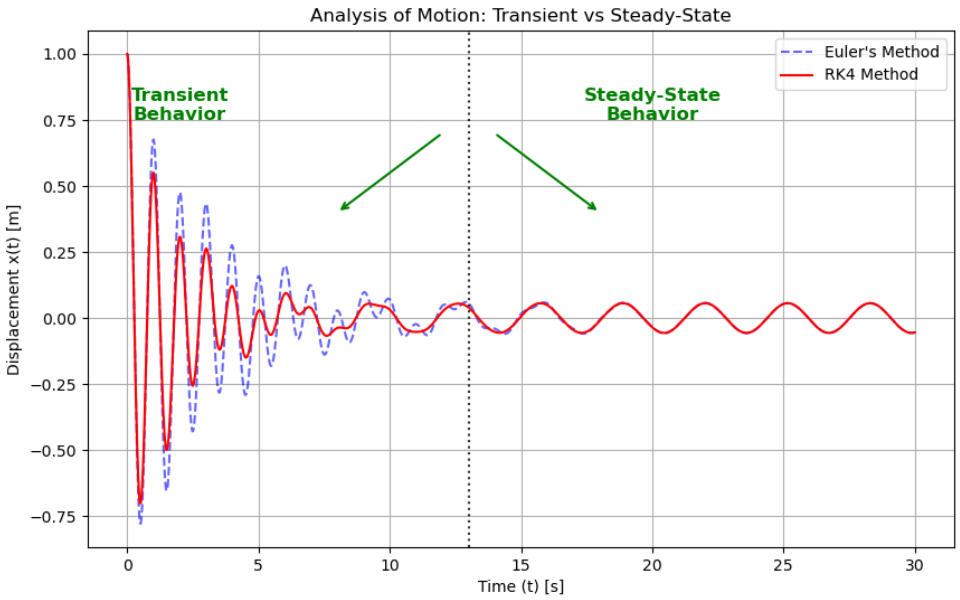

Figure 8: Analysis of Motion: Separation of Transient and Steady-State Behavior. The vertical dotted line at $t = 13$ s marks the approximate end of the transient phase, and start of steady phase.

Based on the plot generated in Part 3 (see Figure 8), we can observe two distinct regions of motion:

- **Transient Behavior ($0 \leq t \lesssim 13$ s):** In the initial phase, the motion is irregular and heavily influenced by the initial conditions ($x(0) = 1.0$, $v(0) = 0.0$). The amplitude and periodicity fluctuate as the system adjusts from its starting state.
- **Steady-State Behavior ($t \gtrsim 13$ s):** After approximately 13 seconds, the initial irregularities have decayed, and the system settles into a regular, periodic sinusoidal motion. In this regime, the oscillation is governed by the external driving force $A_0 \cos(\omega t)$, oscillating at the driving frequency $\omega = 2.0$ rad/s.

Cause of Transient Decay

The governing ordinary differential equation is:

$$\frac{d^2x}{dt^2} + \mathbf{2}\gamma\frac{\mathbf{dx}}{\mathbf{dt}} + \omega_0^2 x = A_0 \cos(\omega t) \tag{50}$$

The term responsible for the decay is the damping term:

$$2\gamma\frac{dx}{dt}$$

Name:	Sricheran Katta	**Assignment No:**	Assignment 4
SR No:	11-01-07-10-93-24-1-25347	**Course Code:**	DS288/ UMC202
Email ID:	sricherank@iisc.ac.in	**Course Name:**	Numerical Methods
Date:	November 17, 2025	**Term:**	AUG 2025

Physical Explanation

For example let's take a swing, now you pull it up and then release it, due to its potential energy it will go backwards and forward. Now the air resistance (the damping term) will remove the energy due to swings initial condition (potential energy) and then the swing will stop eventually. Now if you push the swing every time it comes to you (like the external driving force in the question), then you are giving it energy every time. Initially it will have both the energies, the potential energy of gravity due to initial condition and the energy you are providing in every loop. The energy due to initial condition is the natural rythm(frequency) and the energy you provide causes the driving rythm(frequency). Initially both the rythms clash causing some interference, wobbly motion known as transient phase. Now eventually the potential energy due to initial condition lost due to air resistance causing the natural frequency to die out and then only driving frequency remains. This driving frequency does not die because you are providing energy in every swing and this is energy is cancelled by the air resistance and the swing moves smoothly called steady state phase.

Physically, the transient phase represents a "conflict" between the oscillator's natural frequency (ω_0) and the external driving frequency (ω).

Initially, the system attempts to oscillate at its natural rhythm while being forced to move at the driving frequency. This creates the irregular interference pattern seen in the first 13 seconds (analogous to pushing a swing at the wrong rhythm).

The damping term $2\gamma\frac{dx}{dt}$ represents energy dissipation (friction or air resistance). It continuously removes the energy associated with the initial conditions (the natural oscillation). Eventually, the natural oscillation is completely damped out, and the system "locks in" to the driving frequency, resulting in the smooth steady-state motion.

Mathematical Explanation

Mathematically, the general solution $x(t)$ to this non-homogeneous differential equation is the sum of two parts:

$$x(t) = x_h(t) + x_p(t) \tag{51}$$

- **The Transient Solution (x_h):** This is the solution to the homogeneous equation (setting $A_0\cos(\omega t)$ to zero). It represents the system's natural response and takes the form:

  $$x_h(t) = Ce^{-\gamma t}\cos(\omega_1 t + \phi)$$

  Because of the exponential factor $e^{-\gamma t}$ (where $\gamma = 0.5 > 0$), this term decays to zero as $t \to \infty$.

- **The Steady-State Solution (x_p):** This is the particular solution corresponding to the driving force. It takes the form $x_p(t) = A\cos(\omega t - \delta)$. This term does not decay and persists as long as the driving force is applied.

Therefore, the "wobbly" transient behavior is observed only while $x_h(t)$ is non-negligible. Once $e^{-\gamma t}$ decays effectively to zero (around $t = 13$s), only the steady-state solution $x_p(t)$ remains.

Part 4: Convergence Analysis

We performed convergence analysis to determine the accuracy and order of convergence for both Euler's and RK4 method.

How we did? First, "True Solution" was generated using the RK4 method with an extremely small time step, $\Delta t_{true} = 0.00125$ s.

Next, both Euler's method and the RK4 method were simulated using six different time steps:

$$\Delta t = \{0.0025, 0.005, 0.01, 0.02, 0.04, 0.08\}\ \text{s}$$

For each simulation, the accuracy was quantified using the L2-error (also known as the Root Mean Square Error). This metric provides a single measure of the average deviation of the numerical solution from the true solution over the entire time interval. The formula used is:

$$E_{L2} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_{num}(t_i) - x_{true}(t_i))^2} \tag{52}$$

where:

- N is the total number of time steps compared.
- $x_{num}(t_i)$ is the displacement calculated by the numerical method (Euler or RK4) at time t_i.
- $x_{true}(t_i)$ is the displacement from the high-fidelity true solution at the same time t_i.

Results The L2-errors for each method and time step are presented in Table 5 and plotted on a log-log scale in Figure 9 and also the slopes of the best-fit lines for log-log data. The slopes of the best-fit lines for the log-log data were also computed.

Table 5: L2-Error vs. Time Step

Δt (s)	Euler L2-Error	RK4 L2-Error
0.0025	9.211967e-03	5.160448e-10
0.005	2.084912e-02	8.796614e-09
0.01	4.892623e-02	1.415439e-07
0.02	1.907720e-01	2.274136e-06
0.04	8.510231e+02	3.666689e-05
0.08	8.414367e+11	5.950320e-04

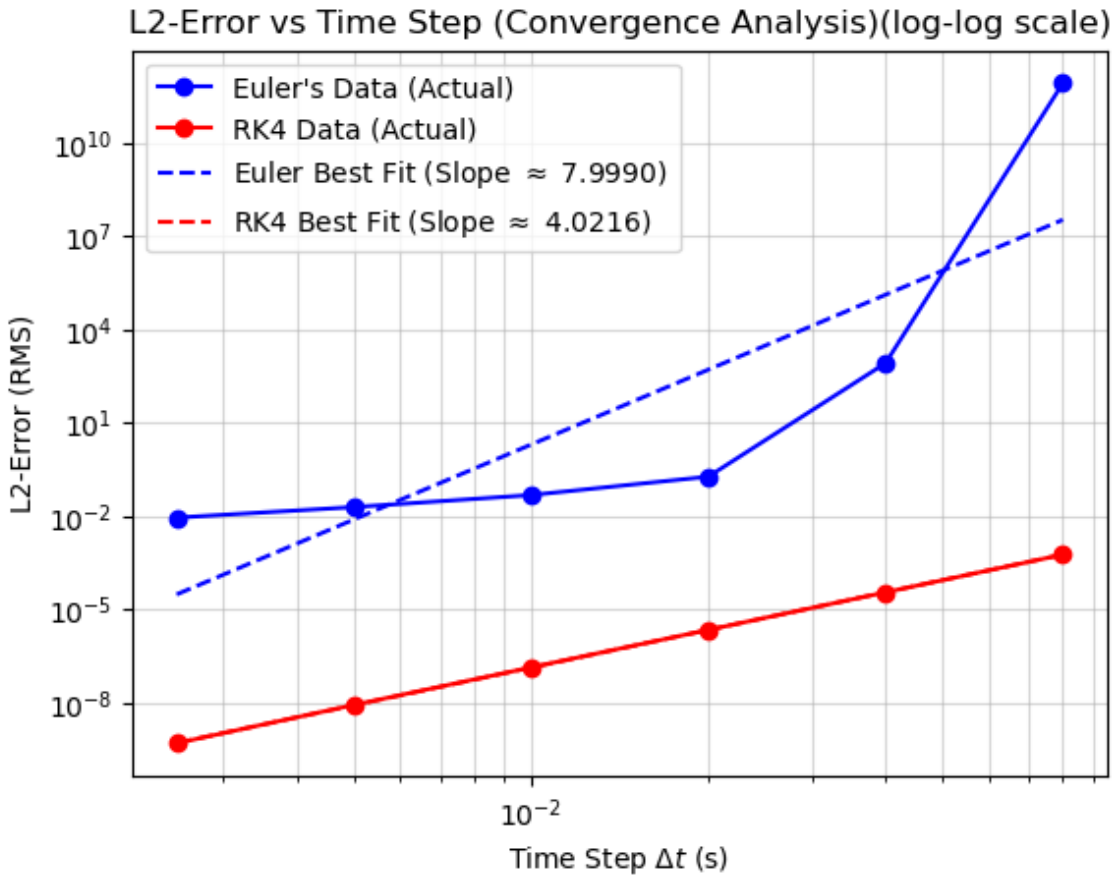

Figure 9: Log-Log plot of L2-Error vs. Time Step (Δt). The slopes represent the calculated order of convergence for each method.

Now the error of the Euler has two parts stable part and unstable parts(exploding errors). This happens for $t = 0.04s$ and $t = 0.08s$ because the time step is two large at it repeatedly overshoots the true solution.

For example, in our case, initially the mass connected to spring is at $x = 1.0m$ and the spring is stretched at maximum and then after small time Δt, the mass moves a tiny bit to the left. Now the spring is slightly les stretched and so the force decreases slightly. This repeats until the spring becomes unstretched ($x = 0m$).

Now what actually happens when Euler is applied with $\Delta t = 0.08s$ is, it calculates a large initial force but incorrectly assumes it remains constant over the entire $\Delta t = 0.08$ s interval. In reality, this force should decrease as the mass moves towards the equilibrium position. By using the maximum force for the whole step, the method calculates a velocity that is far too large, causing the mass to "overshoot" the center and end up at a new position (e.g., $x \approx -1.5$) which is even further from equilibrium. At this new position, the restoring force is now even stronger in the opposite direction. Euler's method repeats the error, applying this new, larger force for the full time step, resulting in an even larger overshoot (e.g., $x \approx +2.5$). This creates a positive feedback loop where the error is amplified exponentially at each step, causing the numerical solution to rapidly "explode" towards infinity.

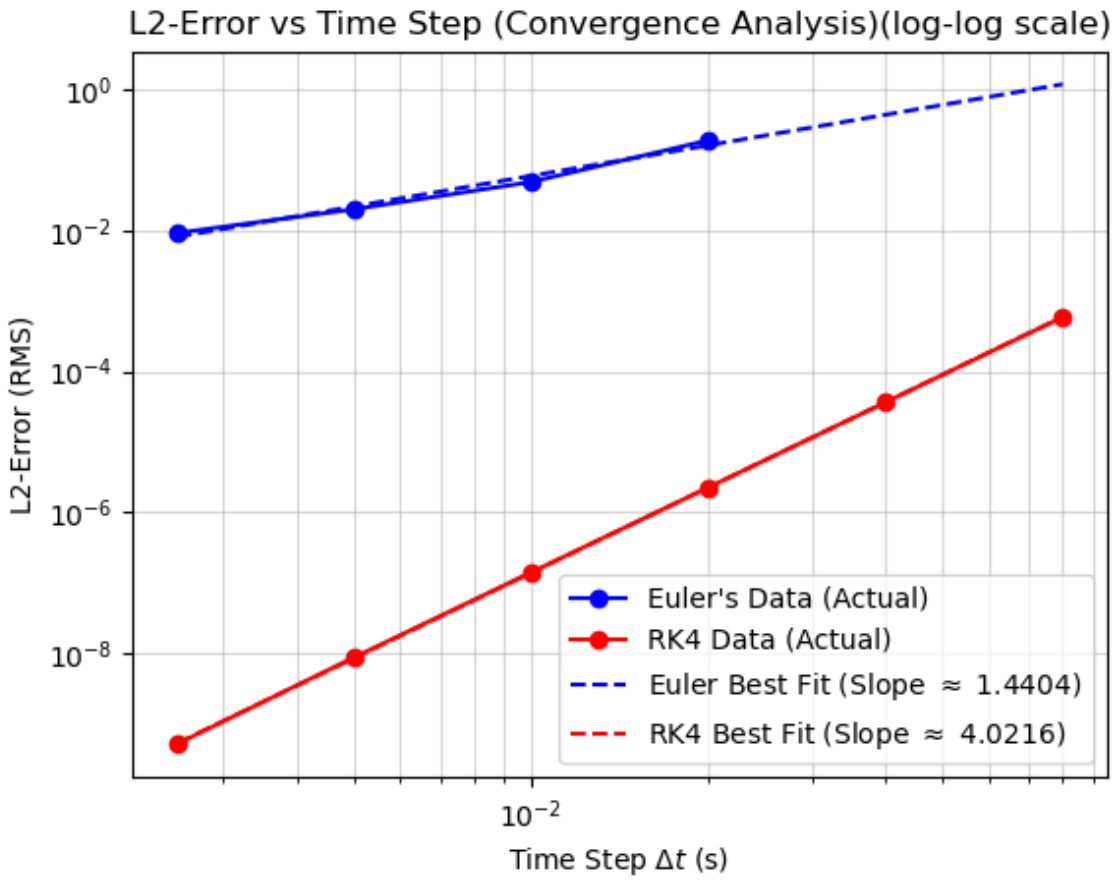

Figure 10: Convergence of Euler's Method in the Stable Region. This plot analyzes only the first four (stable) data points, revealing the true order of convergence for the Euler method.

Analysis of Convergence

The slope of the best-fit line on a log-log plot reveals the method's order of convergence.

RK4 Method: The RK4 method (red line) shows a clear, straight line, indicating consistent convergence. The calculated slope from the linear fit is 4.0216. This is extremely close to the theoretical value of 4. This confirms the RK4 method is 4th-order: its error E is proportional to the step size to the fourth power, $E \propto (\Delta t)^4$.

Euler's Method: The Euler method (blue line) demonstrates numerical instability explained before.

- For small time steps ($\Delta t < 0.04$), the error is stable and proportional to Δt. If we only use these points (second plot), the slope would be ≈ 1.44. Here also the overshooting happens slightly therefore the slope is around 1.44 whereas the theoritical value is 1.
- At $\Delta t = 0.04$, the L2-error suddenly jumps to ≈ 851.

- At $\Delta t = 0.08$, the L2-error explodes to 8.41×10^{11}. This happens because the time step is too large for the method to get accurate results (explained with example before second graph). It repeatedly overshoots the true solution, causing the error to amplify exponentially at each step until the simulation blows up to infinity.
- The calculated slope of ≈ 8.00 is a meaningless value, as it is heavily skewed by these unstable data points. It is not the true order of convergence. The slope from second plot ≈ 1.44 makes sense for the order of convergence and is slightly close to the theoritical value 1.

Conclusion

The analysis confirms that the RK4 method is better for this problem. It is a 4th-order method (slope ≈ 4) and remains stable and accurate across all tested time steps. In contrast, Euler's method is only 1st-order and, more importantly, becomes numerically unstable for larger step sizes, making it completely unreliable for $\Delta t \geq 0.04$ s.

***

References

[1] Díez, Pedro. "A note on the convergence of the secant method for simple and multiple roots." *Applied Mathematics Letters*, vol. 16, no. 8, 2003, pp. 1211–1215.

[2] Atkinson, Kendall E. *An Introduction to Numerical Analysis*. 2nd ed., John Wiley & Sons, 1989.

[3] Muller, David E. "A Method for Solving Algebraic Equations Using an Automatic Computer." *Mathematical Tables and Other Aids to Computation*, vol. 10, no. 56, 1956, pp. 208–215.

[4] Burden, Richard L., and J. Douglas Faires. *Numerical Analysis*. 9th ed., Brooks/Cole, Cengage Learning, 2011.

[5] Ratikanta Behara Sir's class notes